CALL OF ACARISTAN

SOURCE CODE ARCHIVAL MONOLITH

GIT METADATA UNAVAILABLE

DATE: 2026-07-19

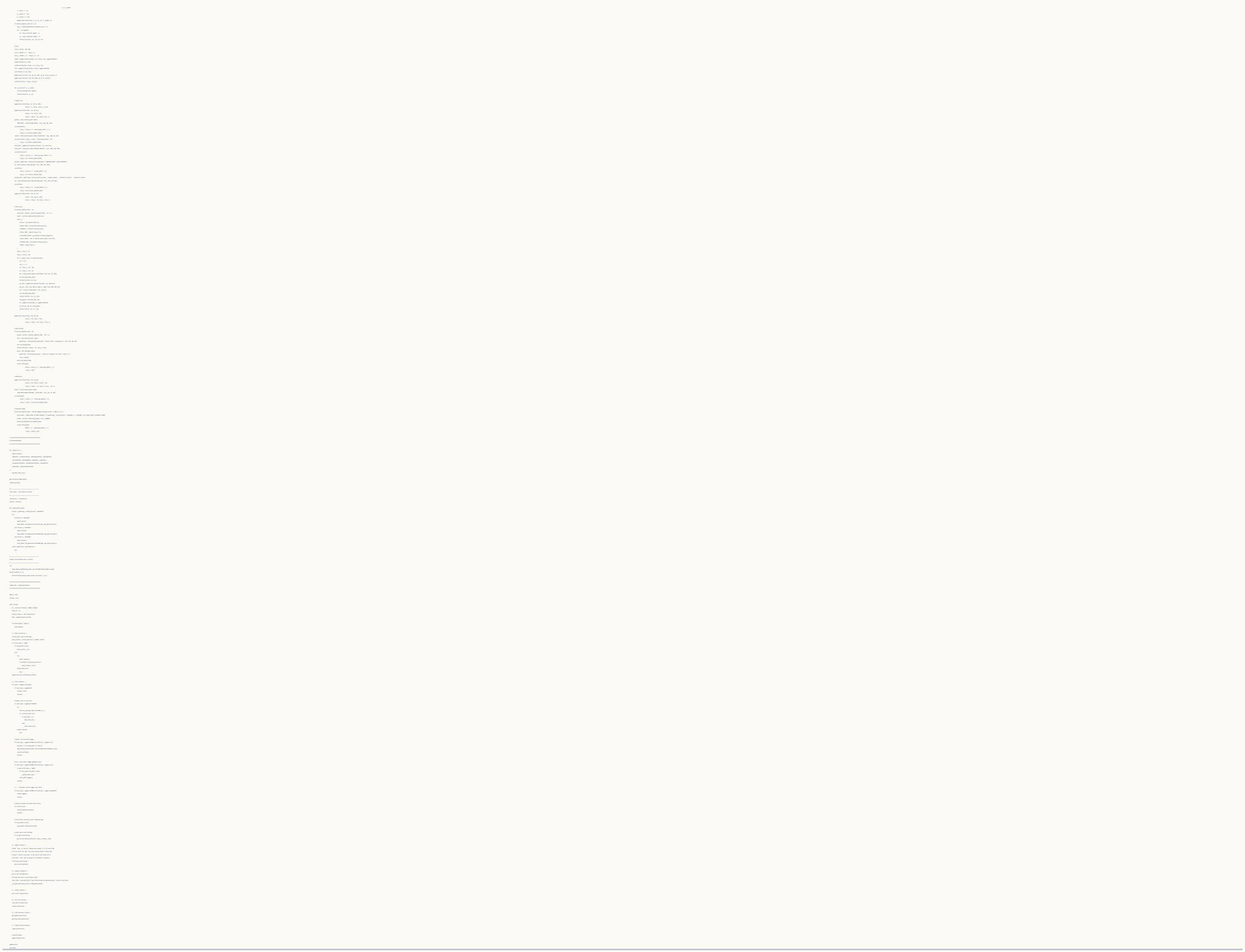
FILES: 32

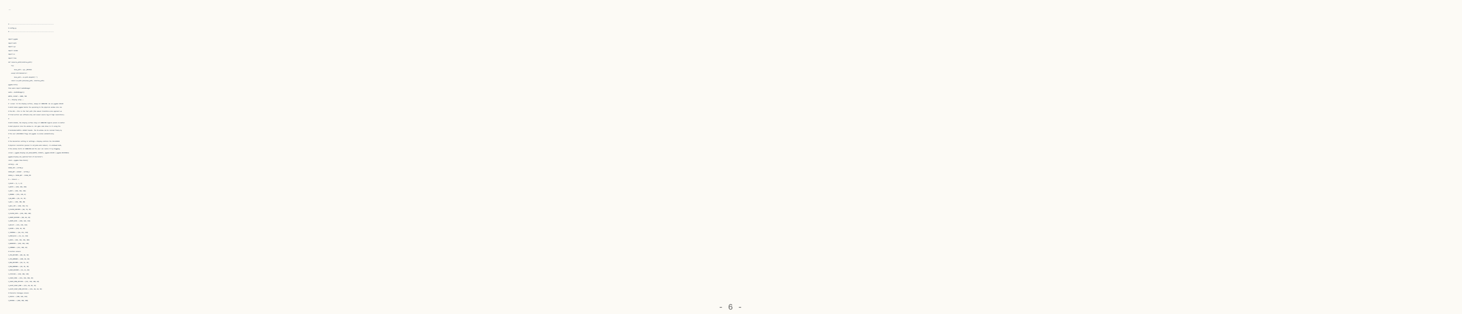
TOTAL LINES: 59,098

DENSITY: EXTREME

GENERATED BY BUILD_CODE_PDF.PY

```
if shot["scene"] in outdoor_cats:
                draw_rain(surface, gs.rain_drops, speed_mult=0)
                surface.blit(_cutscene_fog, (0, 0))
                surface.blit(_cutscene_fog_flip, (0, HEIGHT_H))
            draw_vignette(surface, strength=200)
            draw_scanlines(surface)
            draw_letterbox(surface)
            cur_noframe = (pygame.time.get_ticks() - cs.start_time) >= shot.get("min_ms", 0)
            if not shot.get("hide_caption") and (shot["text"] or shot.get("speaker")):
                gs.draw_dialogue_box(
                    surface,
                    shot["speaker"],
                    shot["text"],
                    cs.char_idx,
                    len(shot["text"]),
                    cur_noframe,
                )
            _g = _advance_grain()
            if _g is not None:
                surface.blit(_g, (0, 0))
        # Shot transition fade
        if abs(cs.transition_timer) > 0:
            fade_alpha = 0
            if cs.transition_timer > 0:
                fade_alpha = int(255 * (cs.transition_timer / 30))
            else:
                fade_alpha = int(255 * (1 - abs(cs.transition_timer) / 30))
            fade_surf = pygame.Surface((WIDTH, HEIGHT))
            fade_surf.fill((0, 0, 0))
            fade_surf.set_alpha(min(255, fade_alpha))
            surface.blit(fade_surf, (0, 0))

class _BriefingState(BaseState):
    name = "BRIEFING"

    def handle_event(self, event, mouse_x, mouse_y, keys):
        if event.type == pygame.KEYDOWN and event.key in (pygame.K_SPACE, pygame.K_RETURN):
            finish_briefing()

    def update(self, dt):
        global briefing_timer
        briefing_timer += 1
        if briefing_timer >= BRIEFING_DURATION:
            finish_briefing()

    def set_ambience(self):
        _set_ambience(["ambient_drone"])

    def render(self, surface):
        draw_briefing_screen()

class _WalkState(BaseState):
    name = "WALK"

    def handle_event(self, event, mouse_x, mouse_y, keys):
        if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
            gsm.transition("PAUSED")
        if event.type == pygame.KEYDOWN and event.key == pygame.K_SPACE:
            if gs.walk_dialogue_active:
                if gs.walk_dialogue_char_idx < len(gs.walk_dialogue_text):
                    gs.walk_dialogue_char_idx = len(gs.walk_dialogue_text)
                else:
                    gs.walk_dialogue_active = False
        if event.type == pygame.KEYDOWN and event.key == pygame.K_e:
            gs.handle_walk_interact()

    def update(self, dt):
        # NOTE: Do NOT update _fixed_cx/_fixed_cy here. The walking cam
        # camera coords don't correspond to the game map, so if PAUSED is
        # entered from WALK, rendering with those coords in game_draw()
        # would draw black void / garbled tiles. Instead, PAUSED from WALK
        # draws via gs.draw_walking_cam().
        _cx, _cy, complete = gs.update_walking_cam(keys)
        if complete:
            start_cutscene(gs.cutscene_shots, "BRIEFING", use_fade=False)
            trigger_transition("CUTSCENE")

    def set_ambience(self):
        _set_ambience(["ambient_drone", "rain", "wind"])

    def render(self, surface):
        gs.draw_walking_cam()
        _g = _advance_grain()
        if _g is not None:
            surface.blit(_g, (0, 0))

class _GameState(_BaseState):
    name = "GAME"

    def handle_event(self, event, mouse_x, mouse_y, keys):
        # Mission 1 interrogation phase
        if getattr(gs, "active_mission", "MISSION1") == "MISSION1":
            _m1_stage = getattr(gs, "mission1_stage", "interrogation")
            _is_escape = (event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE)
            if _m1_stage == "interrogation" and not _is_escape:
                try:
                    import mission1
                    mission1.handle_event(event)
                except ImportError:
                    pass
                return
            elif _is_escape:
                gsm.transition("PAUSED")
                return

        # Mission 2 intel cards
        if getattr(gs, "active_mission", "MISSION1") == "MISSION2" and getattr(gs, "mission2_card", None):
            if event.type == pygame.KEYDOWN and event.key in (pygame.K_e, pygame.K_SPACE, pygame.K_RETURN, pygame.K_ESCAPE):
                gs.handle_interact()
            return

        # Upload UI
        if getattr(gs, "active_mission", "MISSION1") == "MISSION2":
            try:
                import upload_ui
                if upload_ui.is_upload_ui_active():
                    upload_ui.handle_upload_ui_event(event)
                    return
            except ImportError:
                pass

        # Mission 3 crafting overlay
        if getattr(gs, "active_mission", "MISSION1") == "MISSION3":
            try:
                import mission3
                if mission3.is_crafting_active():
                    mission3.handle_crafting_event(event)
                    return
            except ImportError:
                pass

        # Mission 4 outbreak beats lock player actions
        if getattr(gs, "active_mission", "MISSION1") == "MISSION4" and getattr(gs, "block_player_actions", lambda: False)():
            if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
                gsm.transition("PAUSED")
            return

        if gs.active_read_note:
            if event.type == pygame.KEYDOWN and event.key in (pygame.K_SPACE, pygame.K_e, pygame.K_RETURN):
                gs.dismiss_note()
        elif gs.safe_keypad_active:
            if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
                gs.handle_keypad_click(event.pos)
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_ESCAPE:
                    gs.safe_keypad_active = False
                    pygame.mouse.set_visible(False)
                else:
                    gs.handle_keypad_keypress(event.key)
        else:
            if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
                if gs.truck_behaviour:
                    gs.handle_behavior_click(event.pos)
                else:
                    gs.handle_shoot()
            if event.type == pygame.MOUSEBUTTONDOWN and event.button == 3:
                if not gs.truck_behaviour:
                    gs.handle_throw_bottle()
            if event.type == pygame.KEYDOWN and event.key == pygame.K_g:
                gs.handle_throw_bottle()
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                gs.start_reload()
            if event.type == pygame.KEYDOWN and event.key == pygame.K_q:
                gs.switch_weapon()
            if event.type == pygame.KEYDOWN and event.key == pygame.K_e:
                if gs.truck_behaviour:
                    gs.handle_behavior_press()
                else:
                    gs.handle_interact()
            if event.type == pygame.KEYDOWN and event.key == pygame.K_v:
                gs.handle_melee()
            if event.type == pygame.KEYDOWN and event.key == pygame.K_m:
                gs.show_minimap = not gs.show_minimap
            if event.type == pygame.KEYDOWN and event.key == pygame.K_F9:
                gs.debug_jump_to_prfile()
            if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
                gsm.transition("PAUSED")

    def update(self, dt):
        global _fixed_cx, _fixed_cy, mission_complete_saved, mission_complete_timer, mission_complete_fade

        if gs.request_death_screen:
            gs.request_death_screen = False
            if not gs.mission_end_time:
                gs.mission_end_time = pygame.time.get_ticks()
            if hasattr(STATE, "mission_card"):
                STATE.mission_card = None
            gsm.transition("DEATH")
            pygame.mouse.set_visible(True)
            menu.reset_death_animation()
            return

        if gs.request_notebook_inspect:
            gsm.transition("NOTEBOOK_INSPECT")
            gs.request_notebook_inspect = False
            pygame.mouse.set_visible(True)
            import ui_hud; ui_hud.reset_notebook_inspect()
            return

        cx, cy = gs.game_update(keys)
        _fixed_cx, _fixed_cy = cx, cy

        if getattr(gs, "request_mission1_ending", False):
            gs.request_mission1_ending = False
            start_cutscene(gs.mission1_ending_shots, "MISSION1_COMPLETE", use_fade=True)
            trigger_transition("CUTSCENE")
            pygame.mouse.set_visible(True)
        elif getattr(gs, "request_mission2_ending", False):
            gs.request_mission2_ending = False
            start_cutscene(gs.mission2_ending_shots, "MISSION2_COMPLETE", use_fade=True)
            trigger_transition("CUTSCENE")
            pygame.mouse.set_visible(True)
        elif getattr(gs, "request_ruby_pickup_cutscene", False):
            gs.request_ruby_pickup_cutscene = False
            start_cutscene(gs.ruby_pickup_shots, "GAME", use_fade=True)
            trigger_transition("CUTSCENE")
            pygame.mouse.set_visible(True)
        elif getattr(gs, "request_k9_pickup_cutscene", False):
            gs.request_k9_pickup_cutscene = False
            start_cutscene(gs.k9_pickup_shots, "GAME", use_fade=True)
            trigger_transition("CUTSCENE")
            pygame.mouse.set_visible(True)
        elif getattr(STATE, "request_ammo_bags_cutscene", False):
            STATE.request_ammo_bags_cutscene = False
            # Grant two spare mags to whatever McVeal is currently carrying.
            if STATE.current_weapon == "k9_smg":
                STATE.k9_spare_mags = getattr(STATE, "k9_spare_mags", 0) + 2
            elif STATE.current_weapon == "pistol":
                STATE.spare_mags = getattr(STATE, "spare_mags", 0) + 2
            start_cutscene(gs.ammo_bags_shots, "GAME", use_fade=True)
            trigger_transition("CUTSCENE")
            pygame.mouse.set_visible(True)
        elif getattr(gs, "request_escape_cutscene", False):
            gs.request_escape_cutscene = False
            gsm.transition("ESCAPE_CUTSCENE")
            global escape_cutscene_timer
            escape_cutscene_timer = 0
            pygame.mouse.set_visible(False)
        elif getattr(gs, "request_chain_break_cutscene", False):
            gs.request_chain_break_cutscene = False
            # Set state mutations for the breakout.
            STATE.mission4_chain_broken = True
            STATE.prayer_disarmed = False
            STATE.mission4_weapon = "chain_leg"
            start_cutscene(gs.chain_break_shots, "GAME", use_fade=True)
            trigger_transition("CUTSCENE")
            pygame.mouse.set_visible(True)
        elif getattr(STATE, "request_survivor_transition_cutscene", False):
            STATE.request_survivor_transition_cutscene = False
            start_cutscene(gs.survivor_transition_shots, "GAME", use_fade=True)
            trigger_transition("CUTSCENE")
            pygame.mouse.set_visible(True)
        elif getattr(STATE, "request_vent_cutscene", False):
            STATE.request_vent_cutscene = False
            start_cutscene(gs.vent_escape_shots, "GAME", use_fade=True)
            trigger_transition("CUTSCENE")
            pygame.mouse.set_visible(True)
        elif gs.request_mission_complete:
            gs.request_mission_complete = False
            if not gs.mission_end_time:
                gs.mission_end_time = pygame.time.get_ticks()
            gsm.transition("MISSION_COMPLETE")
            mission_complete_timer = 0
            mission_complete_fade = 0

    def set_ambience(self):
        if getattr(gs, "active_mission", "MISSION1") == "MISSION1":
            _set_ambience(["ambient_drone", "rain", "wind"])
        else:
            _set_ambience(self())

    def render(self, surface):
        gs.game_draw(_fixed_cx, _fixed_cy)
        _g = _advance_grain()
        if _g is not None:
            surface.blit(_g, (0, 0))
        gs.draw_read_note()
        gs.draw_safe_keypad()
        import ui_hud as _hud
        try:
            import upload_ui
            if not upload_ui.is_upload_ui_active():
                _hud.draw_gameplay_crosshair()
        except ImportError:
            _hud.draw_gameplay_crosshair()

class _EscapeCutsceneState(BaseState):
    name = "ESCAPE_CUTSCENE"

    def handle_event(self, event, mouse_x, mouse_y, keys):
        global mission_complete_timer, mission_complete_fade
        if event.type == pygame.KEYDOWN and event.key in (pygame.K_SPACE, pygame.K_RETURN):
            gs.mission_end_time = pygame.time.get_ticks()
            gsm.transition("MISSION_COMPLETE")
            mission_complete_timer = 0
            mission_complete_fade = 0
            pygame.mouse.set_visible(True)

    def update(self, dt):
        global escape_cutscene_timer, mission_complete_timer, mission_complete_fade
        escape_cutscene_timer += 1
        if escape_cutscene_timer == 30:
            audio.play("door_open", volume=0.6)
        if escape_cutscene_timer == 90:
            audio.play("rig_low_boom", volume=0.5)
        if escape_cutscene_timer == 180:
            audio.play("distant_howl", volume=0.5)
        if escape_cutscene_timer == 420:
            audio.play("radio_static", volume=0.4)
        if escape_cutscene_timer == 770:
            audio.play("helmet_glitch", volume=0.35)
        if escape_cutscene_timer == 880:
            audio.play("menu_click", volume=0.35)
        if escape_cutscene_timer >= ESCAPE_CUTSCENE_DURATION:
            gs.mission_end_time = pygame.time.get_ticks()
            gsm.transition("MISSION_COMPLETE")
            mission_complete_timer = 0
            mission_complete_fade = 0
            pygame.mouse.set_visible(True)

    def set_ambience(self):
        _set_ambience(["rain", "wind", "ambient_drone"])

    def render(self, surface):
        gs.draw_hotmud_escape_cutscene(surface, escape_cutscene_timer)

class _NotebookInspectState(BaseState):
    name = "NOTEBOOK_INSPECT"

    def handle_event(self, event, mouse_x, mouse_y, keys):
        if event.type == pygame.KEYDOWN and event.key in (pygame.K_SPACE, pygame.K_RETURN):
            gsm.transition("GAME")
            pygame.mouse.set_visible(False)

    def update(self, dt):
        pass

    def set_ambience(self):
        _set_ambience(self())

    def render(self, surface):
        gs.game_draw(_fixed_cx, _fixed_cy)
        gs.draw_notebook_inspection()

class _PausedState(BaseState):
    name = "PAUSED"
    _return_to = "GAME"  # Snapshot of the state we came from

    def enter(self):
        # Snapshot the previous state on entry. This prevents the
        # PAUSED → SETTINGS → PAUSED round-trip from corrupting
        # gsm.previous_name (which would become "SETTINGS" instead
        # of the original gameplay state).
        _PausedState._return_to = gsm.previous_name

    def handle_event(self, event, mouse_x, mouse_y, keys):
        global _settings_return_to
        if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
            gsm.transition(_PausedState._return_to)
            pygame.mouse.set_visible(False)
        if menu.pause_buttons[0].is_clicked(event, (mouse_x, mouse_y)):
            gsm.transition(_PausedState._return_to)
            pygame.mouse.set_visible(False)
        if menu.pause_buttons[1].is_clicked(event, (mouse_x, mouse_y)):
            _settings_return_to = "PAUSED"
            gsm.transition("SETTINGS")
        if menu.pause_buttons[2].is_clicked(event, (mouse_x, mouse_y)):
            gs.restart_current_mission()
            gsm.transition("GAME")
            pygame.mouse.set_visible(False)
            menu.reset_death_animation()
        if menu.pause_buttons[3].is_clicked(event, (mouse_x, mouse_y)):
            trigger_transition("MENU")

    def update(self, dt):
        pass

    def set_ambience(self):
        _set_ambience(self())

    def render(self, surface):
        # Draw the correct background based on where we came from.
        # WALK uses its own renderer; GAME/NOTEBOOK_INSPECT use game_draw.
        if _PausedState._return_to == "WALK":
            gs.draw_walking_cam()
        else:
            gs.game_draw(_fixed_cx, _fixed_cy)
        _g = _advance_grain()
        if _g is not None:
            surface.blit(_g, (0, 0))
        menu.draw_pause_menu()

class _DeathState(BaseState):
    name = "DEATH"

    def handle_event(self, event, mouse_x, mouse_y, keys):
        if (getattr(STATE, "active_mission", "MISSION1") == "MISSION4"
                and getattr(STATE, "m4_black_badger_death_active", False)
                and hasattr(menu, "black_badger_death_buttons_ready")
                and not menu.black_badger_death_buttons_ready()):
            return

        if menu.death_buttons[0].is_clicked(event, (mouse_x, mouse_y)):
            try:
                gs.restart_current_mission()
            except Exception as _restart_err:
                import traceback
                traceback.print_exc()
                # Save the checkpoint flag BEFORE the fallback — start_mission()
                # calls STATE.reset() which clears mission4_checkpoint_reached to
                # False. Without this, a failed checkpoint respawn (e.g.
                # _anybody_badger throws) cascades into a permanent loss of the
                # checkpoint: the fallback does a fresh start_mission() (clearing
                # the flag), and every subsequent death also does a full restart
                # instead of a checkpoint respawn. We restore the flag after the
                # fallback so the next death uses the checkpoint.
                _checkpoint_was_reached = getattr(STATE, "mission4_checkpoint_reached", False)
                try:
                    if getattr(STATE, "active_mission", "MISSION1") == "MISSION4":
                        import mission4
                        # If the checkpoint was reached, retry the checkpoint
                        # respawn before falling back to a full mission restart.
                        # The first failure (inside restart_current_mission →
                        # start_mission4_checkpoint) may have been transient
                        # (audio state, partial initialisation race). The retry
                        # calls start_mission4_checkpoint() which re-runs
                        # start_mission4() (STATE.reset + full init) so STATE
                        # is clean — this isn't retrying on corrupted state.
                        _checkpoint_retry_ok = False
                        if _checkpoint_was_reached:
                            try:
                                mission4.start_mission4_checkpoint(preserve_timer=False)
                                _checkpoint_retry_ok = True
                            except Exception:
                                traceback.print_exc()
                        if not _checkpoint_retry_ok:
                            # Fresh mission restart as last resort. Preserve
                            # the checkpoint flag so the next death respawn
                            # uses the checkpoint instead of restarting the
                            # entire mission from the intro cutscene.
                            mission4.start_mission4(preserve_timer=False)
                            if _checkpoint_was_reached:
                                STATE.mission4_checkpoint_reached = True
                    elif getattr(STATE, "active_mission", "MISSION1") == "MISSION3":
                        import mission3
                        if getattr(STATE, "mission3_cabin_checkpoint_reached", False):
                            mission3.start_mission3_cabin_checkpoint(preserve_timer=False)
                        elif getattr(STATE, "mission3_breakout_checkpoint_reached", False):
                            mission3.start_mission3_breakout_checkpoint(preserve_timer=False)
                        else:
                            mission3.start_mission3(preserve_timer=False)
                    else:
                        import mission1
                        mission1.start_mission1(preserve_timer=False)
                except Exception as _fallback_err:
                    traceback.print_exc()
                    trigger_transition("MENU")
                    menu.reset_death_animation()
                    return
            gsm.transition("GAME")
            pygame.mouse.set_visible(False)
            menu.reset_death_animation()
        elif menu.death_buttons[1].is_clicked(event, (mouse_x, mouse_y)):
            trigger_transition("MENU")
            menu.reset_death_animation()

    def update(self, dt):
        pass

    def set_ambience(self):
        _set_ambience(["heartbeat"])

    def render(self, surface):
        if (getattr(STATE, "active_mission", "MISSION1") == "MISSION4"
                and getattr(STATE, "m4_black_badger_death_active", False)
                and hasattr(menu, "draw_black_badger_death_buttons")):
            # Black Badger death is its own final screen. Do NOT call
            # gs.game_draw() here: that draws the gameplay HUD, minimap,
            # objective, ammo panel, etc. The final animated badger frame
            # becomes the death screen on a clean black-blue background.
            surface.fill((3, 4, 8))
            try:
                import mission4
                mission4._draw_black_badger_death_overlay()
            except Exception:
                # If the custom renderer ever fails, fall back to the normal
                # death screen rather than trapping the player.
                gs.game_draw(_fixed_cx, _fixed_cy)
            menu.draw_black_badger_death_buttons()
            return

        gs.game_draw(_fixed_cx, _fixed_cy)
        _g = _advance_grain()
        if _g is not None:
            surface.blit(_g, (0, 0))
        menu.draw_death_screen()

class _MissionCompleteState(BaseState):
    name = "MISSION_COMPLETE"

    def enter(self):
        global mission_complete_timer, mission_complete_fade, mission_complete_saved
        mission_complete_timer = 0
        mission_complete_fade = 0

    def handle_event(self, event, mouse_x, mouse_y, keys):
        global mission_complete_saved
        if event.type == pygame.KEYDOWN and event.key in (pygame.K_SPACE, pygame.K_RETURN):
            if getattr(gs, "active_mission", "MISSION1") == "MISSION1":
                gs.mission_start_time = pygame.time.get_ticks()
                gs.mission_end_time = 0
                mission_complete_saved = False
                start_cutscene(gs.mission2_intro_shots, "MISSION2", use_fade=False)
                trigger_transition("CUTSCENE")
            elif getattr(gs, "active_mission", "MISSION1") == "MISSION2":
                # Campaign flow: Mission 2 completion should lead into the
                # Mission 3 intro cutscene, not drop directly into gameplay.
                # The cutscene's after_state="GAME" path then transitions
                # cleanly into Mission 3 gameplay once the intro finishes.
                gs.mission_start_time = pygame.time.get_ticks()
                gs.mission_end_time = 0
                mission_complete_saved = False
                gs.start_mission3(preserve_timer=True)
                start_cutscene(gs.mission3_intro_shots, "GAME", use_fade=True)
                trigger_transition("CUTSCENE")
                pygame.mouse.set_visible(True)
            else:
                trigger_transition("MENU")
                pygame.mouse.set_visible(True)

    def update(self, dt):
        pass

    def set_ambience(self):
        _set_ambience(self())
        if mission_complete_timer == 0:
            audio.play("mission_complete", volume=0.6)

    def render(self, surface):
        global mission_complete_timer, mission_complete_fade, mission_complete_saved
        mission_complete_timer += 1
        mission_complete_fade = min(255, mission_complete_timer * 3)
        if not mission_complete_saved:
            report = _mission_report_data(success=True)
            if getattr(gs, "active_mission", "MISSION1") == "MISSION1":
                save_system.save_mission_complete(
                    "MISSION_1",
                    "Mission 1 - Quest at Route 57",
                    report["time_str"],
                    stats=report,
                    flags={"mission1_complete": True, "mission2_complete": True},
                )
            else:
                save_system.save_mission_complete(
                    "MISSION_2",
                    "Mission 2 - Black Notebook",
                    report["time_str"],
                    stats=report,
                    flags={"mission2_complete": True},
                )
            mission_complete_saved = True

        # Background gradient
        for i in range(HEIGHT):
```





```
C_[illegible] = ([illegible])
    C_[illegible] = ([illegible])
    # — Title / Wallet —
    TITLE_SIZE = [illegible]
    # — Enemy AI Tuning —
    ENEMY_DETECT_RANGE = [illegible]
    ENEMY_ATTACK_RANGE = [illegible]
    ENEMY_FIRE_COOLDOWN = [illegible]
    ENEMY_DAMAGE = [illegible]
    ENEMY_BASE_HEALTH = [illegible]
    PATH_RECALC_INTERVAL = [illegible]
    PATH_WAYPOINT_RADIUS = [illegible]
    PATH_STUCK_TIMEOUT = [illegible]
    PIXEL_LUT = [illegible]
    # — Fonts —
    font_title = pygame.font.SysFont("Impact", [illegible])
    font_ui = pygame.font.SysFont("Arial", [illegible], bold=True)
    font_tutorial = pygame.font.SysFont("Consolas", [illegible], bold=True)
    font_tutorial_small = pygame.font.SysFont("Consolas", [illegible])
    font_dialogue = pygame.font.SysFont("Consolas", [illegible], bold=True)
    font_speaker = pygame.font.SysFont("Consolas", [illegible], bold=True)
    font_prompt = pygame.font.SysFont("Consolas", [illegible], bold=True)
    font_junction = pygame.font.SysFont("Impact", [illegible])
    font_junction_sub = pygame.font.SysFont("Consolas", [illegible])
    # — Background Image —
    bg_image_path = resource_path(os.path.join("assets", "bg.jpg"))
    if os.path.exists(bg_image_path):
        try:
            bg_image = pygame.image.load(bg_image_path).convert()
            bg_image = pygame.transform.scale(bg_image, (WIDTH, HEIGHT))
        except Exception as e:
            print(f"Failed to load menu background '{bg_image_path}': {e}")
            bg_image = pygame.Surface((WIDTH, HEIGHT)).convert()
            bg_image.fill(([illegible]))
    else:
        bg_image = pygame.Surface((WIDTH, HEIGHT)).convert()
        bg_image.fill(([illegible]))
    # — Grain —
    def make_grain(w, h):
        s = pygame.Surface((w, h), pygame.SRCALPHA)
        for _ in range([illegible]):
            a = random.randint([illegible])
            s.set_at((random.randint(0, w - 1), random.randint(0, h - 1)), (0, 0, 0, a))
        for _ in range([illegible]):
            a = random.randint([illegible])
            s.set_at(
                (random.randint(0, w - 1), random.randint(0, h - 1)), (255, 255, 255, a)
            )
        return s
    GRAIN_SURFACES = [make_grain(WIDTH, HEIGHT) for _ in range([illegible])]
    grain_idx = 0
    grain_timer = 0
    # — Drawing helpers —
    def draw_sky_gradient(
        surface, top_color, bot_color, y_start=SCREEN_TOP, y_end=SCREEN_BOT
    ):
        for i in range(y_end - y_start):
            t = i / (y_end - y_start)
            r = int(top_color[0] + (bot_color[0] - top_color[0]) * t)
            g = int(top_color[1] + (bot_color[1] - top_color[1]) * t)
            b = int(top_color[2] + (bot_color[2] - top_color[2]) * t)
            pygame.draw.line(surface, (r, g, b), (0, y_start + i), (WIDTH, y_start + i))
    def draw_building(surface, x, ground_y, w, h, base_col, win_col, win_lit_col, seed):
        rng = random.Random(seed)
        shadow = pygame.Surface((w + [illegible], h + [illegible]), pygame.SRCALPHA)
        shadow.fill(([illegible]))
        surface.blit(shadow, (x + [illegible], ground_y - h + [illegible]))
        pygame.draw.rect(surface, base_col, (x, ground_y - h, w, h))
        for row in range(0, h, [illegible]):
            pygame.draw.line(
                surface,
                (
                    max(0, base_col[0] - [illegible]),
                    max(0, base_col[1] - [illegible]),
                    max(0, base_col[2] - [illegible]),
                ),
                (x, ground_y - h + row),
                (x + w, ground_y - h + row),
                1,
            )
        for wy in range(ground_y - h + [illegible], ground_y - [illegible], [illegible]):
            for wx in range(x + [illegible], x + w - [illegible], [illegible]):
                lit = rng.random() < [illegible]
                col = win_lit_col if lit else win_col
                pygame.draw.rect(surface, col, (wx, wy, [illegible]))
                if lit:
                    glow = pygame.Surface(([illegible]), pygame.SRCALPHA)
                    glow.fill((*win_lit_col, [illegible]))
                    surface.blit(glow, (wx - [illegible], wy - [illegible]))
        pygame.draw.rect(
            surface,
            (min(255, base_col[0] + [illegible]), min(255, base_col[1] + [illegible]), min(255, base_col[2] + [illegible])),
            (x, ground_y - h, w, [illegible]),
        )
    def draw_cityscape(surface, ground_y, palette, time_ticks, parallax=0.0):
        buildings_far = [
            [illegible]
        ]
        px_far = parallax * [illegible]
        for bx, bh, bw, bc, wc, wlc, seed in buildings_far:
            draw_building(surface, int(bx - px_far), ground_y, bw, bh, bc, wc, wlc, seed)
        buildings_mid = [
            [illegible]
        ]
        px_mid = parallax * [illegible]
        for bx, bh, bw, bc, wc, wlc, seed in buildings_mid:
            draw_building(surface, int(bx - px_mid), ground_y, bw, bh, bc, wc, wlc, seed)
        lamp_positions = [[illegible]]
        for lx in lamp_positions:
            lx_offset = int(lx - parallax * [illegible])
            pygame.draw.rect(surface, ([illegible]), (lx_offset - [illegible], ground_y - [illegible]))
            pygame.draw.rect(surface, ([illegible]), (lx_offset - [illegible], ground_y - [illegible]))
            halo = pygame.Surface(([illegible]), pygame.SRCALPHA)
            for r in range([illegible]):
                alpha = int([illegible])
                pygame.draw.circle(halo, ([illegible], alpha), ([illegible]), r)
            surface.blit(halo, (lx_offset - [illegible], ground_y - [illegible]))
            pygame.draw.circle(surface, ([illegible]), (lx_offset - [illegible], ground_y - [illegible]), [illegible])
    def draw_rain(surface, drops, speed_mul=1.0):
        for i, (x, y, spd) in enumerate(drops):
            ex = x - int(spd * [illegible])
            ey = y + int(spd * speed_mul)
            pygame.draw.line(surface, C_RAIN, (x, y), (ex, ey), 1)
            drops[i] = (x - 1, ey, spd)
            if ey > SCREEN_BOT:
                drops[i] = (
                    random.randint(0, WIDTH),
                    SCREEN_TOP - random.randint(0, [illegible]),
                    spd,
                )
    def _make_vignette(strength=[illegible]):
        vig = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        cx, cy = WIDTH // 2, (SCREEN_TOP + SCREEN_BOT) // 2
        max_r = int(math.hypot(WIDTH // 2, SCREEN_H // 2)) + [illegible]
        step = [illegible]
        for r in range(max_r, 0, -step):
            t = 1.0 - (r / max_r)
            alpha = int(strength * t * t)
            alpha = min(alpha, strength)
            rw = int(r * [illegible])
            rh = int(r * [illegible])
            rect = (cx - r, cy - rh // 2, rw, rh)
            pygame.draw.ellipse(vig, (0, 0, 0, alpha), rect)
        return vig
    VIGNETTE = _make_vignette([illegible])
    def draw_vignette(surface, strength=[illegible]):
        surface.blit(VIGNETTE, (0, 0))
    def draw_letterbox(surface):
        pygame.draw.rect(surface, (0, 0, 0), (0, 0, WIDTH, LETTER_H))
        pygame.draw.rect(surface, (0, 0, 0), (0, HEIGHT - LETTER_H, WIDTH, LETTER_H))
        pygame.draw.line(surface, ([illegible]), (0, LETTER_H), (WIDTH, LETTER_H), 1)
        pygame.draw.line(
            surface, ([illegible]), (0, HEIGHT - LETTER_H), (WIDTH, HEIGHT - LETTER_H), 1
        )
    def _make_scanlines():
        sl = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        for y in range(SCREEN_TOP, SCREEN_BOT, [illegible]):
            pygame.draw.line(sl, (0, 0, 0, [illegible]), (0, y), (WIDTH, y))
        return sl
    SCANLINES = _make_scanlines()
    def draw_scanlines(surface):
        # Honour the Graphics > Scanlines toggle. Settings are pushed into the
        # shared SETTINGS singleton by [illegible]; this is the single render-side
        # gate that actually turns the CRT overlay off when the user disables it.
        # Late import to avoid a config <-> user_settings cycle at module load time.
        try:
            from user_settings import SETTINGS
            if not SETTINGS.scanlines:
                return
        except Exception:
            pass
        surface.blit(SCANLINES, (0, 0))
    def _make_floor_texture():
        rng = random.Random([illegible])
        ts = [illegible]
        surf = pygame.Surface((ts, ts))
        surf.fill(C_BG_DARK)
        for _ in range([illegible]):
            x, y = rng.randint(0, ts - 1), rng.randint(0, ts - 1)
            v = rng.randint([illegible])
            base = C_BG_DARK
            c = (
                max(0, min(255, base[0] + v)),
                max(0, min(255, base[1] + v)),
                max(0, min(255, base[2] + v)),
            )
            surf.set_at((x, y), c)
        seam_col = (
            max(0, C_BG_DARK[0] - [illegible]),
            max(0, C_BG_DARK[1] - [illegible]),
            max(0, C_BG_DARK[2] - [illegible]),
        )
        pygame.draw.line(surf, seam_col, (0, 0), (ts - 1, 0), 1)
        pygame.draw.line(surf, seam_col, (0, 0), (0, ts - 1), 1)
        for _ in range([illegible]):
            cx = rng.randint([illegible], ts - [illegible])
            cy = rng.randint([illegible], ts - [illegible])
            cr = rng.randint([illegible])
            patch = pygame.Surface((cr * 2, cr * 2), pygame.SRCALPHA)
            pygame.draw.ellipse(
                patch, (0, 0, 0, rng.randint([illegible])), (0, 0, cr * 2, cr * 2)
            )
            surf.blit(patch, (cx - cr, cy - cr))
        for _ in range([illegible]):
            cx = rng.randint([illegible], ts - [illegible])
            cy = rng.randint([illegible], ts - [illegible])
            pts = [(cx, cy)]
            for _ in range(rng.randint([illegible])):
                cx += rng.randint([illegible])
                cy += rng.randint([illegible])
                pts.append((cx, cy))
            crack_col = (
                max(0, C_BG_DARK[0] - [illegible]),
                max(0, C_BG_DARK[1] - [illegible]),
                max(0, C_BG_DARK[2] - [illegible]),
            )
            for i in range(len(pts) - 1):
                pygame.draw.line(surf, crack_col, pts[i], pts[i + 1], 1)
        return surf
    FLOOR_TILE = _make_floor_texture()
    FLOOR_TILE_SIZE = [illegible]
    def _make_grass_floor():
        rng = random.Random([illegible])
        ts = [illegible]
        base = ([illegible])
        surf = pygame.Surface((ts, ts))
        surf.fill(base)
        for _ in range([illegible]):
            x, y = rng.randint(0, ts - 1), rng.randint(0, ts - 1)
            v = rng.randint([illegible])
            c = (
                max(0, min(255, base[0] + v)),
                max(0, min(255, base[1] + v)),
                max(0, min(255, base[2] + v)),
            )
            surf.set_at((x, y), c)
        for _ in range([illegible]):
            px = rng.randint(0, ts - 1)
            py = rng.randint(0, ts - 1)
            pr = rng.randint([illegible])
            puddle = pygame.Surface((pr * 2, pr), pygame.SRCALPHA)
            pygame.draw.ellipse(puddle, ([illegible]), (0, 0, pr * 2, pr))
            surf.blit(puddle, (px - pr, py - pr // 2))
        for _ in range([illegible]):
            sx = rng.randint(0, ts)
            sy = rng.randint(0, ts)
            ex = sx + rng.randint([illegible])
            ey = sy + rng.randint([illegible])
            pygame.draw.line(
                surf,
                (max(0, base[0] - [illegible]), max(0, base[1] - [illegible]), max(0, base[2] - [illegible])),
                (sx, sy),
                (ex, ey),
                1,
            )
        return surf
    GRASS_FLOOR = _make_grass_floor()
    def _make_container_tile(r_or_color, g=None, b=None):
        if isinstance(r_or_color, tuple):
            r, g, b = r_or_color
        else:
            r = r_or_color
        t = pygame.Surface((TILE_SIZE, TILE_SIZE))
        t.fill((r, g, b))
        for row in range(0, TILE_SIZE, [illegible]):
            shade = [illegible] if (row // [illegible]) % 2 == 0 else [illegible]
            c = (
                max(0, min(255, r + shade)),
                max(0, min(255, g + shade)),
                max(0, min(255, b + shade)),
            )
            pygame.draw.line(t, c, (0, row), (TILE_SIZE, row), 1)
        rng = random.Random(hash((r, g, b)))
        for _ in range([illegible]):
            rx = rng.randint(0, TILE_SIZE - [illegible])
            ry = rng.randint(0, TILE_SIZE - [illegible])
            rw = rng.randint([illegible])
            rh = rng.randint([illegible])
            rust = pygame.Surface((rw, rh), pygame.SRCALPHA)
            rust.fill(([illegible]))
            t.blit(rust, (rx, ry))
        for corner_x, corner_y in [
            ([illegible]),
            (TILE_SIZE - [illegible]),
            ([illegible], TILE_SIZE - [illegible]),
            (TILE_SIZE - [illegible], TILE_SIZE - [illegible]),
        ]:
            pygame.draw.circle(
                t,
                (min(255, r + [illegible]), min(255, g + [illegible]), min(255, b + [illegible])),
                (corner_x, corner_y),
                [illegible],
            )
            pygame.draw.circle(
                t,
                (max(0, r - [illegible]), max(0, g - [illegible]), max(0, b - [illegible])),
                (corner_x + 1, corner_y + 1),
                [illegible],
            )
        return t
    def _make_warehouse_wall():
        t = pygame.Surface((TILE_SIZE, TILE_SIZE))
        t.fill(([illegible]))
        rng = random.Random([illegible])
        for _ in range([illegible]):
            x = rng.randint(0, TILE_SIZE - 1)
            y = rng.randint(0, TILE_SIZE - 1)
            v = rng.randint([illegible])
            c = ([illegible] + v, [illegible] + v, [illegible] + v)
            t.set_at((x, y), c)
        for row in range(0, TILE_SIZE, [illegible]):
            shade = [illegible] if (row // [illegible]) % 2 == 0 else [illegible]
            pygame.draw.line(
                t, ([illegible] + shade, [illegible] + shade, [illegible] + shade), (0, row), (TILE_SIZE, row), 1
            )
        pygame.draw.line(
            t, ([illegible]), (0, TILE_SIZE - [illegible]), (TILE_SIZE - [illegible], TILE_SIZE - [illegible]), [illegible]
        )
        for _ in range([illegible]):
            dx = rng.randint(0, TILE_SIZE - 1)
            dy = rng.randint(0, TILE_SIZE - 1)
            pygame.draw.circle(t, ([illegible]), (dx, dy), [illegible])
        return t
    CONTAINER_TILE_RED = _make_container_tile([illegible])
    CONTAINER_TILE_BLUE = _make_container_tile([illegible])
    CONTAINER_TILE_RUST = _make_container_tile([illegible])
    CONTAINER_TILE_GREY = _make_container_tile([illegible])
    WAREHOUSE_WALL = _make_warehouse_wall()
    def _make_muzzle_flash():
        size = [illegible]
        surf = pygame.Surface((size, size), pygame.SRCALPHA)
        cx, cy = size // 2, size // 2
        for r in range([illegible]):
            a = int([illegible] * (1 - r / [illegible]))
            ratio = r / [illegible]
            cr = int(255 * (1 - ratio * [illegible]))
            cg = int([illegible] * (1 - ratio * [illegible]))
            cb = int([illegible] * (1 - ratio * [illegible]))
            pygame.draw.circle(surf, (cr, cg, cb, a), (cx, cy), r)
        pygame.draw.circle(surf, ([illegible]), (cx, cy), [illegible])
        return surf
    MUZZLE_FLASH = _make_muzzle_flash()
    class Particle:
        def __init__(self, x, y, color):
            self.x, self.y = x, y
            angle = random.uniform(0, math.pi * 2)
            speed = random.uniform([illegible])
            self.dx = math.cos(angle) * speed
            self.dy = math.sin(angle) * speed
            self.life = random.randint([illegible])
            self.color = color
            self.size = random.randint([illegible])
        def update(self):
            self.x += self.dx
            self.y += self.dy
            self.dx *= [illegible]
            self.dy *= [illegible]
            self.life -= 1
            self.size = max(1, int(self.life / [illegible]))
        def draw(self, surface, offset_x, offset_y):
            if self.life > 0:
                pygame.draw.rect(
                    surface,
                    self.color,
                    (self.x - offset_x, self.y - offset_y, self.size, self.size),
                )
    class Casing:
        CASING_W, CASING_H = [illegible]
        LIFETIME = [illegible]
        def __init__(self, x, y, eject_angle):
            self.x, self.y = x, y
            spread = random.uniform([illegible])
            speed = random.uniform([illegible])
            self.dx = math.cos(eject_angle + spread) * speed
            self.dy = math.sin(eject_angle + spread) * speed
            self.rot = random.uniform(0, math.pi * 2)
            self.rot_speed = random.uniform([illegible])
            self.life = self.LIFETIME
            self.grounded = False
        def rect(self):
            return pygame.Rect(
                self.x - self.CASING_W // 2,
                self.y - self.CASING_H // 2,
                self.CASING_W,
                self.CASING_H,
            )
        def update(self, walls=()):
            if not self.grounded:
                self.rot += self.rot_speed
                new_x = self.x + self.dx
                new_rect = pygame.Rect(
                    new_x - self.CASING_W // 2,
                    self.y - self.CASING_H // 2,
                    self.CASING_W,
                    self.CASING_H,
                )
                for w in walls:
                    if new_rect.colliderect(w):
                        if self.dx > 0:
                            new_rect.right = w.left
                        elif self.dx < 0:
                            new_rect.left = w.right
                        self.x = new_rect.centerx
                        self.dx *= [illegible]
                        self.rot_speed *= [illegible]
                        break
                else:
                    self.x = new_x
                new_y = self.y + self.dy
                new_rect = pygame.Rect(
                    self.x - self.CASING_W // 2,
                    new_y - self.CASING_H // 2,
                    self.CASING_W,
                    self.CASING_H,
                )
                for w in walls:
                    if new_rect.colliderect(w):
                        if self.dy > 0:
                            new_rect.bottom = w.top
                        elif self.dy < 0:
                            new_rect.top = w.bottom
                        self.y = new_rect.centery
                        self.dy *= [illegible]
                        self.rot_speed *= [illegible]
                        break
                else:
                    self.y = new_y
                self.dx *= [illegible]
                self.dy *= [illegible]
                self.rot_speed *= [illegible]
                if abs(self.dx) < [illegible] and abs(self.dy) < [illegible]:
                    self.dx = 0
                    self.dy = 0
                    self.grounded = True
            self.life -= 1
        def draw(self, surface, offset_x, offset_y):
            if self.life <= 0:
                return
            sx = self.x - offset_x
            sy = self.y - offset_y
            hw = self.CASING_W / 2
            hh = self.CASING_H / 2
            cos_r = math.cos(self.rot)
            sin_r = math.sin(self.rot)
            corners = []
            for lx, ly in ((-hw, -hh), (hw, -hh), (hw, hh), (-hw, hh)):
                rx = lx * cos_r - ly * sin_r
                ry = lx * sin_r + ly * cos_r
                corners.append((sx + rx, sy + ry))
            pygame.draw.polygon(surface, ([illegible]), corners)
            pygame.draw.polygon(surface, ([illegible]), corners, 1)
    # — Door constants —
    C_DOOR = ([illegible])
    C_DOOR_FRAME = ([illegible])
    C_DOOR_HANDLE = ([illegible])
    C_DOOR_OPEN = ([illegible])
    DOOR_OPEN_RANGE = [illegible]
    DOOR_CLOSE_DELAY = [illegible]
    DOOR_ENEMY_DISTANCE = [illegible]
    DOOR_INTERACT_RANGE = [illegible]
    DOOR_TILE = pygame.Surface((TILE_SIZE, TILE_SIZE))
    DOOR_TILE.fill(C_DOOR)
    pygame.draw.rect(DOOR_TILE, C_DOOR_FRAME, (0, 0, TILE_SIZE, TILE_SIZE), [illegible])
    for px in range([illegible], TILE_SIZE - [illegible], [illegible]):
        pygame.draw.line(DOOR_TILE, ([illegible]), (px, [illegible]), (px, TILE_SIZE - [illegible]), 1)
        pygame.draw.line(DOOR_TILE, ([illegible]), (px + 1, [illegible]), (px + 1, TILE_SIZE - [illegible]), 1)
    pygame.draw.line(
        DOOR_TILE, ([illegible]), ([illegible], TILE_SIZE // 2), (TILE_SIZE - [illegible], TILE_SIZE // 2), [illegible]
    )
    pygame.draw.line(
        DOOR_TILE, ([illegible]),
        ([illegible], TILE_SIZE * 3 // 4), (TILE_SIZE - [illegible], TILE_SIZE * 3 // 4), [illegible],
    )
    for cx, cy in [
        ([illegible]), (TILE_SIZE - [illegible]), ([illegible], TILE_SIZE - [illegible]), (TILE_SIZE - [illegible], TILE_SIZE - [illegible]),
    ]:
        pygame.draw.circle(DOOR_TILE, ([illegible]), (cx, cy), [illegible])
        pygame.draw.circle(DOOR_TILE, ([illegible]), (cx + 1, cy + 1), [illegible])
    cx_d, cy_d = TILE_SIZE // 2, TILE_SIZE // 2
    pygame.draw.circle(DOOR_TILE, ([illegible]), (cx_d, cy_d), [illegible])
    pygame.draw.circle(DOOR_TILE, ([illegible]), (cx_d, cy_d), [illegible])
    for angle in ([illegible]):
        rad = math.radians(angle)
        ex = cx_d + int(math.cos(rad) * [illegible])
        ey = cy_d + int(math.sin(rad) * [illegible])
        pygame.draw.line(DOOR_TILE, ([illegible]), (cx_d, cy_d), (ex, ey), [illegible])
    pygame.draw.circle(DOOR_TILE, C_DOOR_HANDLE, (cx_d, cy_d), [illegible])
    pygame.draw.circle(DOOR_TILE, ([illegible]), (cx_d - 1, cy_d - 1), [illegible])
    DOOR_FRAME_TILE = pygame.Surface((TILE_SIZE, TILE_SIZE), pygame.SRCALPHA)
    pygame.draw.line(
        DOOR_FRAME_TILE, (*C_DOOR_FRAME, [illegible]),
        (0, TILE_SIZE - [illegible]), (TILE_SIZE, TILE_SIZE - [illegible]), [illegible],
    )
    pygame.draw.line(DOOR_FRAME_TILE, (*C_DOOR_FRAME, [illegible]), ([illegible], 0), ([illegible], TILE_SIZE), [illegible])
    pygame.draw.line(
        DOOR_FRAME_TILE, (*C_DOOR_FRAME, [illegible]),
        (TILE_SIZE - [illegible], 0), (TILE_SIZE - [illegible], TILE_SIZE), [illegible],
    )
    # — Key Door Tiles —
    KEY_DOOR_BASE = ([illegible])
    KEY_DOOR_ACCENT = ([illegible])
    KEY_DOOR_DARK = ([illegible])
    KEY_DOOR_FRAME_COL = ([illegible])
    KEY_DOOR_TILE = pygame.Surface((TILE_SIZE, TILE_SIZE))
    KEY_DOOR_TILE.fill(KEY_DOOR_BASE)
    for row in range(0, TILE_SIZE, [illegible]):
        shade = [illegible] if (row // [illegible]) % 2 == 0 else [illegible]
        c = (
            max(0, KEY_DOOR_BASE[0] + shade),
            max(0, KEY_DOOR_BASE[1] + shade),
            max(0, KEY_DOOR_BASE[2] + shade),
        )
        pygame.draw.line(KEY_DOOR_TILE, c, (0, row), (TILE_SIZE - 1, row), 1)
    for col in ([illegible], TILE_SIZE - [illegible]):
        pygame.draw.line(KEY_DOOR_TILE, KEY_DOOR_ACCENT, (col, [illegible]), (col, TILE_SIZE - [illegible]), [illegible])
    for ry in ([illegible], TILE_SIZE - [illegible]):
        pygame.draw.line(KEY_DOOR_TILE, KEY_DOOR_DARK, ([illegible], ry), (TILE_SIZE - [illegible], ry), [illegible])
    handle_x, handle_y = TILE_SIZE - [illegible], TILE_SIZE // 2
    pygame.draw.rect(
        KEY_DOOR_TILE, ([illegible]), (handle_x - [illegible], handle_y - [illegible]), border_radius=[illegible]
    )
    pygame.draw.circle(KEY_DOOR_TILE, ([illegible]), (handle_x + [illegible], handle_y), [illegible])
    KEY_DOOR_FRAME_TILE = pygame.Surface((TILE_SIZE, TILE_SIZE), pygame.SRCALPHA)
    pygame.draw.line(
        KEY_DOOR_FRAME_TILE, (*KEY_DOOR_FRAME_COL, [illegible]),
        (0, TILE_SIZE - [illegible]), (TILE_SIZE, TILE_SIZE - [illegible]), [illegible],
    )
    pygame.draw.line(
        KEY_DOOR_FRAME_TILE, (*KEY_DOOR_FRAME_COL, [illegible]), ([illegible], 0), ([illegible], TILE_SIZE), [illegible]
    )
    pygame.draw.line(
        KEY_DOOR_FRAME_TILE, (*KEY_DOOR_FRAME_COL, [illegible]),
        (TILE_SIZE - [illegible], 0), (TILE_SIZE - [illegible], TILE_SIZE), [illegible],
    )
    for track_x in ([illegible], TILE_SIZE - [illegible]):
        pygame.draw.line(
            KEY_DOOR_FRAME_TILE, (*KEY_DOOR_ACCENT, [illegible]),
            (track_x, 0), (track_x, TILE_SIZE), [illegible],
```

```
            return mission4.start_mission4_camo_checkpoint(preserve_timer=preserve_timer)

def start_mission4_archives_checkpoint(preserve_timer=False):
    """Restart Mission 4 at the archives wing (Phase 4)."""
    return mission4.start_mission4_archives_checkpoint(preserve_timer=preserve_timer)

def restart_current_mission():
    """Restart whichever playable mission is currently active.

    Uses STATE.active_mission to decide — EXCEPT when:
    1. The player is in the walking sim (scratcher.game_state == "walk"),
       which is a transition scene between M3 and M4 and doesn't reset
       active_mission. A restart from walk always goes to Mission 4.
    2. The player has reached the Mission 3 evidence-phase checkpoint
       (STATE.mission3_checkpoint_reached == True). In this case we respawn
       at the shelter with the evidence phase active, instead of restarting
       the entire mission from the intro cutscene. This makes the difficult
       3-wave cleanup team fight feel fair — dying doesn't lose 10+ minutes
       of progress.
    """
    if _is_mission3_active() and not _in_walking_sim():
        if getattr(STATE, "mission3_checkpoint_reached", False):
            return start_mission3_checkpoint(preserve_timer=False)
        return mission3.start_mission3(preserve_timer=False)
    if _is_mission4_active() and not _in_walking_sim():
        if getattr(STATE, "mission4_archives_checkpoint_reached", False):
            return mission4.start_mission4_archives_checkpoint(preserve_timer=False)
        if getattr(STATE, "mission4_camo_checkpoint_reached", False):
            return mission4.start_mission4_camo_checkpoint(preserve_timer=False)
        if getattr(STATE, "mission4_breakout_checkpoint_reached", False):
            return mission4.start_mission4_breakout_checkpoint(preserve_timer=False)
        return mission4.start_mission4(preserve_timer=False)
    return missions.start_mission(preserve_timer=False)

def _in_walking_sim():
    """Returns True if the player is currently in the walking sim transition.

    The walking sim (scratcher.game_state == "walk") sits between Mission 3
    and Mission 4. It doesn't reset STATE.active_mission, so we can't use
    active_mission alone to decide what to restart — we have to check the
    top-level game state too.

    Late-imports scratcher to avoid a circular dependency (scratcher imports
    game_core at module load time). In the running game scratcher is already
    in sys.modules, so this is a dict lookup, not a re-execution.
    """
    try:
        scratcher_mod = sys.modules.get("scratcher") or sys.modules.get("__main__")
        return getattr(scratcher_mod, "game_state", "") == "walk"
    except Exception:
        return False

def game_update(keys):
    if _is_mission3_active():
        return mission3.game_update(keys)
    if _is_mission4_active():
        return mission4.game_update(keys)
    return missions.game_update(keys)

def game_draw(fire, fly):
    if _is_mission3_active():
        return mission3.game_draw(fire, fly)
    if _is_mission4_active():
        return mission4.game_draw(fire, fly)
    return missions.game_draw(fire, fly)

def handle_interact():
    if _is_mission3_active():
        # M3: E key at the drain triggers the crafting minigame (if the player
        # has a chain_key and hasn't sharpened it yet).
        import mission3
        return mission3.handle_interact()
    if _is_mission4_active():
        return mission4.handle_interact()
    return interactions.handle_interact()

def debug_jump_to_m4fit():
    if _is_mission3_active():
        # No debug jump for M3 — it's a linear dialogue mission.
        return
    if _is_mission4_active():
        return mission4.debug_jump_to_evidence()
    return missions.debug_jump_to_m4fit()

class _GameCoreModule(types.ModuleType):
    """Proxy module — routes all game_core globals to new modules/STATE."""

    def __getattr__(self, name):
        if name in _MODULE_FIRST_NAMES:
            for mod in _MODULES:
                if hasattr(mod, name):
                    return getattr(mod, name)
        if hasattr(STATE, name):
            return getattr(STATE, name)
        for mod in _MODULES:
            if hasattr(mod, name):
                return getattr(mod, name)
        raise AttributeError(f"module 'game_core' has no attribute {name!r}")

    def __setattr__(self, name, value):
        wrote = False
        if hasattr(STATE, name):
            setattr(STATE, name, value)
            wrote = True
        # Important: propagate to sub-modules that imported colour/config globals.
        for mod in _MODULES:
            if hasattr(mod, name):
                setattr(mod, name, value)
                wrote = True
        if not wrote:
            super().__setattr__(name, value)

_proxy = _GameCoreModule(__name__)
_proxy.__dict__.update({
    "__doc__": __doc__,
    "__file__": __file__,
    "__package__": __package__,
    "__loader__": __loader__,
    "__spec__": __spec__,
    # Explicit routed functions override same-named functions in split modules.
    "start_mission": start_mission,
    "start_mission2": start_mission2,
    "start_mission2_sniper": start_mission2_sniper,
    "start_mission3_checkpoint": start_mission3_checkpoint,
    "start_mission4": start_mission4,
    "start_mission4_breakout_checkpoint": start_mission4_breakout_checkpoint,
    "start_mission4_camo_checkpoint": start_mission4_camo_checkpoint,
    "start_mission4_archives_checkpoint": start_mission4_archives_checkpoint,
    "restart_current_mission": restart_current_mission,
    "game_update": game_update,
    "game_draw": game_draw,
    "handle_interact": handle_interact,
    "debug_jump_to_m4fit": debug_jump_to_m4fit,
})
sys.modules[__name__] = _proxy
```

```
# ------------------------------------------------------------
# player.py
# ------------------------------------------------------------

# ------------------------------------------------------------
# player.py
# ------------------------------------------------------------

# ------------------------------------------------------------

"""Player character: movement, drawing, weapon rig."""
import math
import time
import pygame
from config import (
    TILE_SIZE, C_PLAYER_UNIFORM, C_PLAYER_SKIN,
)
# Module-level STATE import. Required by receive_melee() below (it reads
# STATE.melee_damage via getattr). Previously this import was missing — the
# original code relied on a lazy `from state import STATE` inside Player
# methods (e.g. barrel_tip), but receive_melee() is a module-level function
# that references STATE directly, causing a NameError on the first melee
# attack. No circular-import risk: state.py only imports pygame.
from state import STATE

_player_shadow = pygame.Surface((24, 10), pygame.SRCALPHA)
pygame.draw.ellipse(_player_shadow, (0, 0, 0, 70), (0, 0, 24, 10))

class Player:
    def __init__(self, x, y, max_hp=100):
        self.rect = pygame.Rect(x, y, 24, 24)
        self.speed = 5
        self.angle = 0
        self.hp = max_hp
        self.max_hp = max_hp
        self.breathing = 0
        self.blink_timer = 0
        self.weapon_raised = False
        self.walk_cycle = 0.0
        self.was_moving = False
        self._recoil_timer = 0
        self._recoil_offset = 0.0
        self._stride_timer = 0
        # ── GDD05 I-frames (Pass 4) ──────────────────────────
        # Per reference/game-design.md "Invincibility frames": after taking
        # any damage, the player is invincible for IFRAME_DURATION seconds
        # and the sprite blinks on/off at ~60ms windows so the player can
        # see that they were hit and that they are currently safe.
        # update_iframes(dt) must be called every frame from each mission's
        # game_update() to advance the timer + blink.
        self.invincible_timer = 0.0
        self.visible = True
        # IFRAME_DURATION is the total invincibility window in seconds.
        # 0.75s matches the GDD05 reference value and feels right for a
        # tactical shooter where the player needs time to reposition after
        # a hit. Tune via the constant below — never inline a magic number.
        self.IFRAME_DURATION = 0.75
        # BLINK_PERIOD_MS drives the sprite-flash cadence. 60ms = 4 frames
        # at 60 FPS, so the sprite blinks ~6 times during the 0.75s window.
        self.BLINK_PERIOD_MS = 60

    def take_damage(self, amount, force=False):
        """Apply damage to the player, respecting the i-frame window.

        Per GDD05 reference/game-design.md: Returns True if damage was
        applied, False if blocked by i-frames. Callers may use the return
        value to decide whether to spawn blood particles / floating text
        / trigger hit-stop (a hit absorbed by i-frames should NOT produce
        a feedback burst, otherwise the player thinks they were hit).

        `force=True` bypasses i-frames — use ONLY for set-piece damage
        that the player cannot dodge and that the narrative requires to
        land (e.g. the Mission 2 sniper intro shot, the Sentori blast
        during a scripted trigger). Tactical combat damage must always
        go through the default path so i-frames can do their job.
        """
        if not force and self.invincible_timer > 0:
            return False
        self.hp -= amount
        self.invincible_timer = self.IFRAME_DURATION
        # Force the first blink frame to be invisible so the hit is
        # immediately readable (otherwise the first 60ms shows the sprite
        # normally and the player doesn't notice i-frames started).
        self.visible = False
        return True

    def update_iframes(self, dt):
        """Advance the i-frame timer + sprite blink. Call every frame.

        Per GDD05 spec: visible = (int(timer_ms) // BLINK_PERIOD_MS) % 2 == 0.
        When the timer expires, visibility is restored to True.
        """
        if self.invincible_timer > 0:
            self.invincible_timer -= dt
            if self.invincible_timer <= 0:
                self.invincible_timer = 0.0
                self.visible = True
            else:
                # Blink: visible only on alternating ~BLINK_PERIOD_MS windows.
                timer_ms = int(self.invincible_timer * 1000)
                self.visible = (timer_ms // self.BLINK_PERIOD_MS) % 2 == 0
        else:
            self.visible = True

    def move(self, dx, dy, walls):
        self.rect.x += dx
        for wall in walls:
            if self.rect.colliderect(wall):
                if dx > 0:
                    self.rect.right = wall.left
                if dx < 0:
                    self.rect.left = wall.right
        self.rect.y += dy
        for wall in walls:
            if self.rect.colliderect(wall):
                if dy > 0:
                    self.rect.bottom = wall.top
                if dy < 0:
                    self.rect.top = wall.bottom

    def barrel_tip(self):
        """World-space muzzle position — matches the gun drawing proportions."""
        from state import STATE
        a = self.angle

        current_weapon = getattr(STATE, "current_weapon", "pistol")
        if current_weapon == "sk16":
            arm_len = 16
            grip_offset = 4
            gun_len = 26
            muzzle_extra = 3
        elif current_weapon == "hk_smg":
            arm_len = 16
            grip_offset = 4
            gun_len = 20
            muzzle_extra = 5  # compact suppressor
        else:
            arm_len = 16
            grip_offset = 4
            gun_len = 17
            muzzle_extra = 2

        reach = arm_len + grip_offset + gun_len + muzzle_extra
        return (
            self.rect.centerx + math.cos(a) * reach,
            self.rect.centery + math.sin(a) * reach,
        )

    def draw(
        self,
        surface,
        offset_x,
        offset_y,
        walls=(),
        reload_active=False,
        reload_progress=0.0,
        melee_active=False,
        melee_progress=0.0,
        no_gun=False,
        is_moving=False,
    ):
        global STATE
        draw_x = self.rect.centerx - offset_x
        draw_y = self.rect.centery - offset_y

        if is_moving and not self.was_moving:
            self.walk_cycle = 0.0
        self.was_moving = is_moving

        self.breathing = math.sin(time.time() * 0.3) * 1.0
        draw_y += self.breathing

        # -- walk bob --
        if is_moving:
            self.walk_cycle += 0.15
        else:
            self.walk_cycle *= 0.85
        sway_offset = 0.0

        # -- Recoil --
        if self._recoil_timer > 0:
            self._recoil_timer -= 1
            self._recoil_offset = -3.0 * (self._recoil_timer / 6.0)
        else:
            self._recoil_offset *= 0.6

        if self._stride_timer > 0:
            self._stride_timer -= 1
        self.blink_timer += 1
        blinking = 2 if (180 < self.blink_timer < 186) else 0
        if self.blink_timer > 186:
            self.blink_timer = 0
        a = self.angle
        d_x = math.cos(a)
        d_y = math.sin(a)
        perp_x = math.cos(a + math.pi / 2)
        perp_y = math.sin(a + math.pi / 2)

        arm_angle = a
        if is_moving:
            # weapon hand lags behind direction changes
            arm_angle += sway_offset * 0.05
        if reload_active:
            from state import STATE
            p = reload_progress
            _cw_reload = getattr(STATE, "current_weapon", "pistol")
            _is_sk16_reload = _cw_reload == "sk16"
            _is_hk_reload = _cw_reload == "hk_smg"

            if _is_sk16_reload:
                # SK-16 mag-swap reload:
                # 0.00-0.30: rifle tilts, mag drops
                # 0.30-0.60: hold empty
                # 0.60-1.00: new mag locks in, rifle returns
                if p < 0.30:
                    t = p / 0.30
                    arm_factor = 1.0 - t * 0.25
                    arm_angle = a + t * 0.35
                elif p < 0.60:
                    t = (p - 0.30) / 0.30
                    arm_factor = 0.75
                    tremor = math.sin(p * math.pi * 8) * 0.03
                    arm_angle = a + 0.35 + tremor
                else:
                    t = (p - 0.60) / 0.40
                    ease = 1.0 - (1.0 - t) * (1.0 - t)
                    arm_factor = 0.75 + ease * 0.25
                    arm_angle = a + (0.35 * (1.0 - ease))
            elif _is_hk_reload:
                # H-K suppressed SMG reload — compact 25ms-frame insert,
                # stick mag strips out, palm-slap seats the fresh mag, then
                # # the muzzle snaps back up. More mechanical than the pistol,
                # less heavy than the SK-16.
                if p < 0.25:
                    t = p / 0.25
                    arm_factor = 1.0 - t * 0.20
                    arm_angle = a + t * 0.40
                elif p < 0.50:
                    t = (p - 0.25) / 0.25
                    arm_factor = 0.80
                    arm_angle = a + 0.40 + math.sin(t * math.pi * 4) * 0.025
                elif p < 0.70:
                    t = (p - 0.50) / 0.20
                    arm_factor = 0.80 + t * 0.08
                    arm_angle = a + 0.40 - t * 0.10
                else:
                    t = (p - 0.70) / 0.30
                    ease = 1.0 - (1.0 - t) * (1.0 - t)
                    arm_factor = 0.88 + ease * 0.12
                    arm_angle = a + 0.30 * (1.0 - ease)
            else:
                # Pistol reload (unchanged)
                if p < 0.25:
                    arm_factor = 1.0 - (p / 0.25) * 0.3
                elif p < 0.75:
                    arm_factor = 0.7
                    wobble = math.sin(p * math.pi * 8) * 0.08
                    arm_angle = a + wobble
                else:
                    t = (p - 0.75) / 0.25
                    arm_factor = 0.7 + t * 0.3
                    wobble = math.sin(p * math.pi * 8) * 0.08 * (1.0 - t)
                    arm_angle = a + wobble
        elif melee_active:
            p = melee_progress
            current_weapon_for_melee = getattr(STATE, "current_weapon", "pistol")
            if current_weapon_for_melee == "hk_smg":
                # H-K melee: a compact buttstroke / muzzle shove. It should
                # feel different from the pistol slap and from big-swing melee:
                # pull the gun across the body, snap forward, then settle.
                if p < 0.20:
                    t = p / 0.20
                    arm_factor = 0.88
                    arm_angle = a + 0.18 * t
                elif p < 0.58:
                    t = (p - 0.20) / 0.38
                    punch = 1.0 - (1.0 - t) * (1.0 - t)
                    arm_factor = 0.88 + punch * 0.22
                    arm_angle = a + 0.18 - punch * 0.24
                else:
                    t = (p - 0.58) / 0.42
                    arm_factor = 1.10 - t * 0.10
                    arm_angle = a - 0.06 * (1.0 - t)
            else:
                if p < 0.25:
                    t = p / 0.25
                    arm_factor = 1.0 - t * 0.4
                    arm_angle = a - t * math.pi / 4
                elif p < 0.75:
                    t = (p - 0.25) / 0.5
                    arm_factor = 0.6 + t * 0.8
                    arm_angle = a - math.pi / 4 + t * (math.pi / 4 + math.pi / 3)
                else:
                    t = (p - 0.75) / 0.25
                    arm_factor = 1.4 - t * 0.4
                    arm_angle = a + math.pi / 3 - t * math.pi / 3
        else:
            arm_factor = 1.0

        surface.blit(_player_shadow, (int(draw_x - 12), int(draw_y - 4)))
        # ── GDD05 I-frame blink (Pass 4) ──────────────────────
        # Per reference/game-design.md: while invincible_timer > 0, the
        # sprite blinks on/off at ~60ms windows. The shadow is ALWAYS drawn
        # so the player can still read their position; only the body blinks.
        # All per-frame state above (breathing, walk_cycle, recoil, blink)
        # has already advanced by this point, so timing stays correct.
        if not self.visible:
            return
        body_w, body_h = 20, 14
        cos_b, sin_b = math.cos(a), math.sin(a)
        torso_pts = []
        for bx, by in [
            (-body_w // 2, -body_h // 2),
            (body_w // 2, -body_h // 2),
            (body_w // 2, body_h // 2),
            (-body_w // 2, body_h // 2),
        ]:
            rx = bx * cos_b - by * sin_b
            ry = bx * sin_b + by * cos_b
            torso_pts.append((draw_x + rx, draw_y + ry))
        pygame.draw.polygon(surface, C_PLAYER_UNIFORM, torso_pts)
        if self.max_hp > 100:
            vest_w, vest_h = 16, 10
            vest_pts = []
            for bx, by in [
                (-vest_w // 2, -vest_h // 2),
                (vest_w // 2, -vest_h // 2),
                (vest_w // 2, vest_h // 2),
                (-vest_w // 2, vest_h // 2),
            ]:
                rx = bx * cos_b - by * sin_b
                ry = bx * sin_b + by * cos_b
                vest_pts.append((draw_x + rx, draw_y + ry))
            pygame.draw.polygon(surface, (50, 54, 58), vest_pts)
            pygame.draw.polygon(surface, (30, 30, 34), vest_pts, 1)
            for off in (-4, 4):
                sx1 = draw_x + perp_x * off - d_x * 3
                sy1 = draw_y + perp_y * off - d_y * 3
                sx2 = draw_x + perp_x * off + d_x * 3
                sy2 = draw_y + perp_y * off + d_y * 3
                pygame.draw.line(surface, (20, 22, 26), (sx1, sy1), (sx2, sy2), 1)
            ridge_w, ridge_h = 6, 8
            ridge_pts = []
            for bx, by in [
                (-ridge_w // 2, -ridge_h // 2),
                (ridge_w // 2, -ridge_h // 2),
                (ridge_w // 2, ridge_h // 2),
                (-ridge_w // 2, ridge_h // 2),
            ]:
                rx = bx * cos_b - by * sin_b
                ry = bx * sin_b + by * cos_b
                ridge_pts.append((draw_x + rx, draw_y + ry))
            pygame.draw.polygon(surface, (40, 44, 48), ridge_pts)
        pygame.draw.line(
            surface,
            (
                C_PLAYER_UNIFORM[0] + 10,
                C_PLAYER_UNIFORM[1] + 10,
                C_PLAYER_UNIFORM[2] + 8,
            ),
            (draw_x - d_x * 5, draw_y - d_y * 5),
            (draw_x + d_x * 5, draw_y + d_y * 5),
            4,
        )
        for off in (-6, 6):
            sx1 = draw_x + perp_x * off - d_x * 3
            sy1 = draw_y + perp_y * off - d_y * 3
            sx2 = draw_x + perp_x * off + d_x * 3
            sy2 = draw_y + perp_y * off + d_y * 3
            pygame.draw.line(
                surface,
                (
                    C_PLAYER_UNIFORM[0] - 8,
                    C_PLAYER_UNIFORM[1] - 5,
                    C_PLAYER_UNIFORM[2] - 5,
                ),
                (sx1, sy1),
                (sx2, sy2),
                2,
            )

        foot_arm_len = int(16 * arm_factor)
        foot_arm_dx = math.cos(arm_angle)
        foot_arm_dy = math.sin(arm_angle)
        foot_grip_x = draw_x + foot_arm_dx * (foot_arm_len + 4)
        foot_grip_y = draw_y + foot_arm_dy * (foot_arm_len + 4)

        foot_gun_end_x = foot_grip_x + foot_arm_dx * 24
        foot_gun_end_y = foot_grip_y + foot_arm_dy * 24

        min_t = float("inf")
        if not no_gun and walls:
            for wall in walls:
                wr = pygame.Rect(
                    wall.x - offset_x, wall.y - offset_y, TILE_SIZE, TILE_SIZE
                )
                clipped = wr.clipline(
                    (foot_grip_x, foot_grip_y), (foot_gun_end_x, foot_gun_end_y)
                )
                if clipped:
                    cx, cy = clipped[0]
                    t = (cx - foot_grip_x) * foot_arm_dx + (
                        cy - foot_grip_y
                    ) * foot_arm_dy
                    if t < min_t:
                        min_t = t

        raise_factor = 0.0
        if not no_gun and min_t < 24:
            raise_factor = max(0.0, min(1.0, (24.0 - min_t) / 24.0))

        # -- weapon swap animation: lower the gun fully at midpoint, raise back up --
        from state import STATE
        swap_timer = getattr(STATE, "weapon_swap_timer", 0)
        swap_duration = getattr(STATE, "weapon_swap_duration", 24)
        if swap_timer > 0 and swap_duration > 0:
            # Progress: 0.0 -> 1.0 across the full swap
            progress = 1.0 - (swap_timer / swap_duration)
            # Triangle wave: 0 at start, 1 at midpoint, 0 at end
            if progress < 0.5:
                swap_factor = progress * 2.0         # lowering (0 -> 1)
            else:
```

```
                                    pygame.draw.line(screen, (40, 44, 48), (ct_x - 8, ct_y + 14), (ct_x + 14, ct_y + 14), 1)
    # exact contact point marker on top of the fingertip
    pygame.draw.circle(screen, C_ORANGE, (ct_x, ct_y), 3, 1)
else:
    # Ranked reticle: Gated on SETTINGS.show_crosshair so the user's
    # "Crosshair: OFF" preference is honored. Respects crosshair_size
    # (1=small, 2=medium, 3=large) and colorblind_mode (recolors the
    # reticle for accessibility) — same logic as the shared
    # ui_hud.draw_gameplay_crosshair(), so the two renderers match.
    if _show_ch:
        from user_settings import SETTINGS
        cb = SETTINGS.colorblind_mode
        if cb == "protanopia":
            col = (90, 180, 255)  # blue reads well for red-deficient vision
        elif cb == "deuteranopia":
            col = (255, 220, 80)  # yellow
        elif cb == "tritanopia":
            col = (255, 100, 140)  # pinkish-red
        else:
            col = C_ORANGE
        size = max(1, int(SETTINGS.crosshair_size))
        ring_r = size * 8
        arm = size * 6
        gap = max(3, size // 2)
        pygame.draw.circle(screen, col, (ct_x, ct_y), ring_r, 1)
        pygame.draw.circle(screen, col, (ct_x, ct_y), max(1, size // 2))
        for x1, y1, x2, y2 in (
            (ct_x - arm, ct_y, ct_x - gap, ct_y),
            (ct_x + gap, ct_y, ct_x + arm, ct_y),
            (ct_x, ct_y - arm, ct_x, ct_y - gap),
            (ct_x, ct_y + gap, ct_x, ct_y + arm),
        ):
            pygame.draw.line(screen, col, (x1, y1), (x2, y2), 1)
if STATE.muzzle_flash_timer > 0:
    if STATE.player:
        if hasattr(STATE.player, "barrel_tip"):
            tip_x, tip_y = STATE.player.barrel_tip()
        else:
            tip_x = STATE.player.rect.centerx + math.cos(STATE.player.angle) * 30
            tip_y = STATE.player.rect.centery + math.sin(STATE.player.angle) * 30
        flash_x = tip_x - fcx
        flash_y = tip_y - fcy
    else:
        flash_x, flash_y = ct_x, ct_y
    screen.blit(MUZZLE_FLASH, (int(flash_x - 20), int(flash_y - 20)))
    STATE.muzzle_flash_timer -= 1
if STATE.player_dead:
    # Death animation overlay: blood pool, fallen body, weapon eject,
    # tint/blur. The camera tilt is applied at the very end of the frame
    # (after HUD) so the entire screen rotates including UI.
    import death_anim
    death_anim.draw_death_overlay(screen, fcx, fcy)
if STATE.gl_flash_timer > 0 and not getattr(STATE, "reduced_motion", False):
    flash_surf = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    flash_surf.fill((255, 255, 255, min(255, STATE.gl_flash_timer * 25)))
    screen.blit(flash_surf, (0, 0))
# -- Breach stage flicker --
if STATE.breach_stage in (1, 2):
    if (pygame.time.get_ticks() // 60) % 3 == 0:
        flicker = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        flicker.fill((0, 0, 0, random.randint(20, 60)))
        screen.blit(flicker, (0, 0))
draw_scanlines(screen)
draw_vignette(screen)
if STATE.player and not STATE.truck_boarding:
    hb_x, hb_y, hb_w, hb_h = 30, HEIGHT - 50, 240, 18
    hp_ratio = max(0.0, STATE.player.hp / STATE.player.max_hp)
    hud_bg = pygame.Surface((hb_w + 20, hb_h + 40), pygame.SRCALPHA)
    pygame.draw.rect(hud_bg, (0, 0, 0, 160), (0, 0, hb_w + 20, hb_h + 40))
    screen.blit(hud_bg, (hb_x - 10, hb_y - 30))
    screen.blit(
        font_terminal_small.render("HP", True, C_TERMINAL), (hb_x, hb_y - 20)
    )
    pygame.draw.rect(screen, (30, 30, 30), (hb_x, hb_y, hb_w, hb_h))
    if hp_ratio > 0.6:
        bar_col = (int(200 * hp_ratio), int(200 * hp_ratio), 40)
    elif hp_ratio > 0.25:
        bar_col = (200, int(200 * hp_ratio * 2), 40)
    else:
        bar_col = (200, 40, 40)
    pygame.draw.rect(screen, bar_col, (hb_x, hb_y, int(hb_w * hp_ratio), hb_h))
    pygame.draw.rect(screen, C_TERMINAL, (hb_x, hb_y, hb_w, hb_h), 1)
    # Objective text (without "OBJECTIVE:" prefix — the shared helper
    # prepends it, matching RO's behavior).
    obj_text = "Locate hitmen in the warehouse"
    if STATE.mission_phase == "firefight":
        if STATE.hostages_rescued:
            obj_text = "Reach safety"
        elif len(STATE.enemies) > 0:
            obj_text = f"Clear hostiles ({len(STATE.enemies)} remaining)"
        else:
            obj_text = "Rescue the hostage"
    elif STATE.mission_phase == "escape":
        obj_text = "Reach safe zone"
    draw_objective_panel(screen, obj_text)
    ammo_x, ammo_y = 30, hb_y - 70
    bg_w, bg_h = 300, 50
    bg = pygame.Surface((bg_w, bg_h), pygame.SRCALPHA)
    pygame.draw.rect(bg, (0, 0, 0, 140), (0, 0, bg_w, bg_h), border_radius=6)
    pygame.draw.rect(bg, (80, 80, 90), (0, 0, bg_w, bg_h), 1, border_radius=6)
    screen.blit(bg, (ammo_x - 10, ammo_y))
    for i in range(3):
        mx = ammo_x + 8 + i * 26
        my = ammo_y + 8
        is_active = i == 0
        if is_active:
            bullets_fill = min(STATE.ammo_in_mag, 10)
            is_empty = STATE.ammo_in_mag == 0
        else:
            bullets_fill = 10 if STATE.spare_mags >= i else 0
            is_empty = STATE.spare_mags < i
        top_w, bot_w, h = 16, 20, 34
        taper = (top_w - bot_w) // 2
        body_pts = [
            (mx, my),
            (mx + top_w, my),
            (mx + taper + bot_w, my + h),
            (mx + taper, my + h),
        ]
        if is_empty:
            base = (35, 30, 30)
            hi = (55, 50, 48)
            rim = (18, 16, 15)
        elif is_active:
            base = (60, 90, 70)
            hi = (110, 140, 100)
            rim = (70, 44, 30)
        else:
            base = (50, 48, 44)
            hi = (60, 80, 80)
            rim = (40, 34, 30)
        pygame.draw.polygon(screen, base, body_pts)
        pygame.draw.polygon(
            screen,
            hi,
            [
                (mx + 2, my + 2),
                (mx + 5, my + 2),
                (mx + taper + 5, my + h - 2),
                (mx + taper + 2, my + h - 2),
            ],
        )
        outline = (
            C_TERMINAL if is_active else (rim if not is_empty else (50, 30, 30))
        )
        pygame.draw.polygon(screen, outline, body_pts, 1)
        bullet_y = my + 8
        gold = (200, 160, 40)
        dark = (30, 32, 40) if not is_empty else (60, 30, 30)
        for bi in range(10):
            col = gold if bi < bullets_fill else dark
            pygame.draw.circle(screen, col, (mx + top_w // 2, bullet_y + bi * 3), 1)
        if is_empty:
            pygame.draw.line(
                screen,
                (90, 30, 30),
                (mx + 2, my + 2),
                (mx + top_w - 2, my + h - 2),
                1,
            )
            pygame.draw.line(
                screen,
                (90, 30, 30),
                (mx + top_w - 2, my + 2),
                (mx + 2, my + h - 2),
                1,
            )
    ammo_text = f"{STATE.ammo_in_mag}/{STATE.mag_size}"
    text_col = C_TERMINAL if STATE.ammo_in_mag > 0 else (200, 60, 60)
    screen.blit(
        font_terminal_small.render(ammo_text, True, text_col),
        (ammo_x + 110, ammo_y + 8),
    )
    screen.blit(
        font_terminal_small.render("AMMO", True, (160, 160, 160)),
        (ammo_x + 110, ammo_y + 26),
    )
    if STATE.is_reloading:
        bar_x, bar_y = ammo_x + 160, ammo_y + 20
        bar_w, bar_h = 80, 8
        progress = 1.0 - STATE.reload_timer / 60.0
        pygame.draw.rect(
            screen, (40, 50, 60), (bar_x, bar_y, bar_w, bar_h), border_radius=4
        )
        pygame.draw.rect(
            screen,
            (100, 200, 255),
            (bar_x, bar_y, int(bar_w * progress), bar_h),
            border_radius=4,
        )
        reload_txt = font_terminal_small.render("RELOADING", True, (100, 200, 255))
        screen.blit(reload_txt, (ammo_x + 160, ammo_y + 30))
    else:
        mag_label = f"MAGS: {STATE.spare_mags}"
        screen.blit(
            font_terminal_small.render(mag_label, True, (160, 160, 160)),
            (ammo_x + 160, ammo_y + 20),
        )
    # -- Bottles HUD --
    bot_x = ammo_x - 6
    bot_y = ammo_y - 40
    bot_bg_w, bot_bg_h = 150, 34
    bot_bg = pygame.Surface((bot_bg_w, bot_bg_h), pygame.SRCALPHA)
    pygame.draw.rect(bot_bg, (0, 0, 0, 140), (0, 0, bot_bg_w, bot_bg_h), border_radius=6)
    pygame.draw.rect(bot_bg, (80, 80, 90), (0, 0, bot_bg_w, bot_bg_h), 1, border_radius=6)
    screen.blit(bot_bg, (bot_x, bot_y))
    for bi in range(3):
        icon_x = bot_x + 10 + bi * 22
        icon_y = bot_y + 17
        _draw_bottle_icon(screen, icon_x, icon_y, bi < STATE.player_bottles)
    bot_lbl = font_terminal_small.render(
        f"BOTTLES  [Q]", True,
        (220, 200, 80) if STATE.player_bottles > 0 else (100, 90, 80),
    )
    screen.blit(bot_lbl, (bot_x + 100, bot_y + 4))
    bot_count = font_terminal_small.render(
        f"{STATE.player_bottles} / 3", True,
        C_TERMINAL if STATE.player_bottles > 0 else (100, 60, 60),
    )
    screen.blit(bot_count, (bot_x + 100, bot_y + 18))
    host_bg2 = pygame.Surface((180, 24), pygame.SRCALPHA)
    pygame.draw.rect(host_bg2, (0, 0, 0, 140), (0, 0, 180, 24))
    screen.blit(host_bg2, (30, 80))
    screen.blit(
        font_terminal_small.render(f"HOSTILES: {len(STATE.enemies)}", True, C_TERMINAL),
        (36, 84),
    )
if STATE.show_minimap and STATE.minimap:
    STATE.minimap.draw(
        screen,
        STATE.player,
        STATE.enemies,
        fcx,
        fcy,
        STATE.extraction_pos,
        STATE.alarm_pos,
        STATE.hostage_pos,
        STATE.commander_pos,
    )
# -- Death animation: camera tilt --
# Apply at the very end of the frame so the entire rendered scene
# (world + HUD + minimap) rotates together. No-op when inactive.
if STATE.player_dead:
    import death_anim
    death_anim.apply_camera_tilt(screen)
```

```
----


# ----------------------------------------------
# cutscenes.py
# ----------------------------------------------

# ----------------------------------------------
# cutscenes.py
# ----------------------------------------------

# cutscenes.py
# ----------------------------------------------

"""Cutscene visuals and dialogue: character drawings, scene backgrounds, shot FX etc."""
import math
import random
import pygame
from config import *
def draw_soldier(surface, cx, foot_y, scale=1.0, facing_right=True):
    d = int(facing_right) * 2 - 1
    s = scale
    head_r = int(10 * s)
    torso_h = int(34 * s)
    torso_w = int(20 * s)
    leg_h = int(30 * s)
    leg_w = int(10 * s)
    arm_w = int(6 * s)
    arm_h = int(26 * s)
    foot_l = (cx - int(10 * s), foot_y)
    foot_r = (cx + int(10 * s), foot_y)
    hip_y = foot_y - leg_h
    sho_y = hip_y - torso_h
    head_y = sho_y - head_r - int(4 * s)
    shadow = pygame.Surface((int(60 * s), int(14 * s)), pygame.SRCALPHA)
    shadow.fill((0, 0, 0, 60))
    surface.blit(shadow, (cx - int(30 * s), foot_y - int(6 * s)))
    leg_col = (40, 50, 40)
    for lx in (cx - int(6 * s), cx + int(6 * s)):
        pygame.draw.rect(surface, leg_col, (lx - leg_w // 2, hip_y, leg_w, leg_h))
    shoe_col = (30, 30, 30)
    pygame.draw.ellipse(
        surface,
        shoe_col,
        (foot_l[0] - int(10 * s), foot_y - int(4 * s), int(14 * s), int(6 * s)),
    )
    pygame.draw.ellipse(
        surface,
        shoe_col,
        (foot_r[0] - int(4 * s), foot_y - int(4 * s), int(14 * s), int(6 * s)),
    )
    coat_col = (50, 60, 48)
    vest_col = (180, 190, 40)
    vest_pts = [
        (cx - int(torso_w // 2 + 4) * d, sho_y),
        (cx + int(torso_w // 2 + 4) * d, sho_y),
        (cx + int(torso_w // 2 + 8) * d, hip_y),
        (cx - int(torso_w // 2 + 8) * d, hip_y),
    ]
    pygame.draw.polygon(
        surface,
        (int(vest_col[0] * 0.6), int(vest_col[1] * 0.6), int(vest_col[2] * 0.6)),
        vest_pts,
    )
    coat_pts = [
        (cx - torso_w // 2, sho_y),
        (cx + torso_w // 2, sho_y),
        (cx + torso_w // 2 + int(4 * s), hip_y),
        (cx - torso_w // 2 - int(4 * s), hip_y),
    ]
    pygame.draw.polygon(surface, coat_col, coat_pts)
    pygame.draw.line(
        surface,
        (30, 40, 30),
        (cx, sho_y),
        (cx - int(4 * s) * d, sho_y + int(20 * s)),
        int(2 * s),
    )
    arm_side_x = cx + int(torso_w // 2) * d
    back_arm_x = cx - int(torso_w // 2) * d
    pygame.draw.rect(
        surface, (50, 60, 48), (back_arm_x - arm_w // 2, sho_y, arm_w, arm_h)
    )
    pygame.draw.rect(
        surface, (50, 60, 48), (arm_side_x - arm_w // 2, sho_y, arm_w, arm_h)
    )
    pygame.draw.circle(
        surface,
        (int(vest_col[0] * 0.6), int(vest_col[1] * 0.6), int(vest_col[2] * 0.6)),
        (cx + int(vest_col[0] * 0.05 * d), head_y),
        head_r + int(2 * s),
    )
    skin_col = (60, 38, 34)
    pygame.draw.circle(surface, skin_col, (cx, head_y), head_r)
    pygame.draw.arc(
        surface,
        (0, 0, 0),
        (cx - head_r, head_y - head_r, head_r * 2, head_r * 2),
        0,
        math.pi,
        int(head_r * 0.45),
    )
    eye_x = cx + int(4 * s) * d
    eye_y = head_y - int(2 * s)
    pygame.draw.circle(surface, (200, 30, 30), (eye_x, eye_y), int(2 * s))
    pygame.draw.circle(
        surface, (255, 80, 80), (eye_x + int(1 * s) * d, eye_y - int(1 * s)), int(1 * s)
    )
    cig_x = cx + int(14 * s) * d
    cig_y = head_y + int(4 * s)
    pygame.draw.line(
        surface,
        (240, 240, 240),
        (cig_x, cig_y),
        (cig_x + int(6 * s) * d, cig_y - int(2 * s)),
        int(2 * s),
    )
    if int(pygame.time.get_ticks() / 150) % 2 == 0:
        pygame.draw.circle(
            surface,
            (255, 80, 40),
            (cig_x + int(6 * s) * d, cig_y - int(2 * s)),
            int(2 * s),
        )
    ember = pygame.Surface((int(10 * s), int(10 * s)), pygame.SRCALPHA)
    pygame.draw.circle(ember, (255, 100, 40, 60), (int(5 * s), int(5 * s)), int(5 * s))
    surface.blit(ember, (cig_x + int(6 * s) * d, cig_y - int(6 * s)))
def draw_zombie(surface, cx, foot_y, scale=1.0, facing_right=True):
    d = int(facing_right) * 2 - 1
    s = scale
    head_r = int(11 * s)
    torso_h = int(34 * s)
    torso_w = int(24 * s)
    leg_h = int(30 * s)
    leg_w = int(10 * s)
    hip_y = foot_y - leg_h
    sho_y = hip_y - torso_h
    head_y = sho_y - head_r - int(3 * s)
    shadow = pygame.Surface((int(60 * s), int(14 * s)), pygame.SRCALPHA)
    shadow.fill((0, 0, 0, 60))
    surface.blit(shadow, (cx - int(30 * s), foot_y - int(6 * s)))
    leg_col = (35, 40, 50)
    for lx in (cx - int(6 * s), cx + int(6 * s)):
        pygame.draw.rect(surface, leg_col, (lx - leg_w // 2, hip_y, leg_w, leg_h))
    pygame.draw.ellipse(
        surface,
        (25, 25, 25),
        (cx - int(16 * s), foot_y - int(4 * s), int(14 * s), int(6 * s)),
    )
    pygame.draw.ellipse(
        surface,
        (25, 25, 25),
        (cx + int(2 * s), foot_y - int(4 * s), int(14 * s), int(6 * s)),
    )
    suit_col = C_DARK_BROWN
    coat_pts = [
        (cx - torso_w // 2, sho_y),
        (cx + torso_w // 2, sho_y),
        (cx + torso_w // 2 + int(6 * s), hip_y),
        (cx - torso_w // 2 - int(6 * s), hip_y),
    ]
    pygame.draw.polygon(surface, suit_col, coat_pts)
    pygame.draw.line(
        surface,
        C_DARK_BLUE,
        (cx - int(10 * s), sho_y + int(10 * s)),
        (cx + int(10 * s), sho_y + int(10 * s)),
        int(2 * s),
    )
    for i in range(3):
        px = cx + int((torso_w // 2 - 6 - i * 5) * d)
        pygame.draw.circle(
            surface, (200, 180, 60), (px, sho_y + int(8 * s)), int(2 * s)
        )
    arm_col = suit_col
    pygame.draw.rect(
        surface,
        arm_col,
        (cx - int(torso_w // 2) * d - int(4 * s), sho_y, int(8 * s), int(26 * s)),
    )
    pygame.draw.rect(
        surface,
        arm_col,
        (cx + int(torso_w // 2) * d - int(4 * s), sho_y, int(8 * s), int(26 * s)),
    )
    pygame.draw.circle(surface, (200, 160, 130), (cx, head_y), head_r)
    pygame.draw.arc(
        surface,
        (40, 30, 20),
        (cx - head_r, head_y - head_r, head_r * 2, head_r * 2),
        math.pi * 0.1,
        math.pi * 0.9,
        4,
    )
    cig_x = cx + int(10 * s) * d
    cig_y = head_y + int(4 * s)
    pygame.draw.line(
        surface,
        (240, 240, 240),
        (cig_x, cig_y),
        (cig_x + int(6 * s) * d, cig_y - int(2 * s)),
        int(2 * s),
    )
    if int(pygame.time.get_ticks() / 150) % 2 == 0:
        pygame.draw.circle(
            surface,
            (255, 80, 40),
            (cig_x + int(6 * s) * d, cig_y - int(2 * s)),
            int(2 * s),
        )
    eye_x = cx + int(5 * s) * d
    eye_y = head_y - int(2 * s)
    pygame.draw.circle(surface, (30, 30, 30), (eye_x, eye_y), int(2 * s))
def draw_civilian(
    surface, cx, foot_y, scale=1.0, facing_right=True, is_silhouette=False
):
    d = int(facing_right) * 2 - 1
    s = scale
    head_r = int(10 * s)
    torso_h = int(30 * s)
    torso_w = int(20 * s)
    leg_h = int(30 * s)
    leg_w = int(10 * s)
    hip_y = foot_y - leg_h
    sho_y = hip_y - torso_h
    head_y = sho_y - head_r - int(3 * s)
    shadow = pygame.Surface((int(60 * s), int(14 * s)), pygame.SRCALPHA)
    shadow.fill((0, 0, 0, 60))
    surface.blit(shadow, (cx - int(30 * s), foot_y - int(6 * s)))
    if is_silhouette:
        for lx in (cx - int(6 * s), cx + int(6 * s)):
            pygame.draw.rect(
                surface, (8, 8, 10), (lx - leg_w // 2, hip_y, leg_w, leg_h)
            )
        coat_pts = [
            (cx - torso_w // 2, sho_y),
            (cx + torso_w // 2, sho_y),
            (cx + torso_w // 2 + int(4 * s), hip_y),
            (cx - torso_w // 2 - int(4 * s), hip_y),
        ]
        pygame.draw.polygon(surface, (8, 8, 10), coat_pts)
        pygame.draw.rect(
            surface,
            (8, 8, 10),
            (cx - int(14 * s), sho_y + int(10 * s), int(28 * s), int(14 * s)),
        )
        pygame.draw.circle(surface, (8, 8, 10), (cx, head_y), head_r)
        pygame.draw.arc(
            surface,
            (8, 8, 10),
            (
                cx - head_r - int(2 * s),
                head_y - head_r - int(2 * s),
                (head_r + int(2 * s)) * 2,
                head_r + int(6 * s),
            ),
            0,
            math.pi,
            int(head_r * 0.6),
        )
        return
    leg_col = (30, 35, 40)
    for lx in (cx - int(6 * s), cx + int(6 * s)):
        pygame.draw.rect(surface, leg_col, (lx - leg_w // 2, hip_y, leg_w, leg_h))
    for lx in (cx - int(6 * s), cx + int(6 * s)):
        pygame.draw.rect(
            surface,
            (80, 140, 40),
            (lx - leg_w // 2, hip_y + int(10 * s), leg_w, int(10 * s)),
        )
    pygame.draw.ellipse(
        surface,
        (50, 40, 40),
        (cx - int(16 * s), foot_y - int(4 * s), int(14 * s), int(6 * s)),
    )
    pygame.draw.ellipse(
        surface,
        (50, 40, 40),
        (cx + int(2 * s), foot_y - int(4 * s), int(14 * s), int(6 * s)),
    )
    coat_col = (80, 140, 40)
    coat_pts = [
        (cx - torso_w // 2, sho_y),
        (cx + torso_w // 2, sho_y),
        (cx + torso_w // 2 + int(4 * s), hip_y),
        (cx - torso_w // 2 - int(4 * s), hip_y),
    ]
    pygame.draw.polygon(surface, coat_col, coat_pts)
    vest_col = (40, 70, 30)
    pygame.draw.rect(
        surface,
        vest_col,
        (cx - int(10 * s), sho_y + int(10 * s), int(20 * s), int(14 * s)),
    )
    for i in range(3):
        px = cx - int(10 * s) + i * int(10 * s)
        pygame.draw.rect(
            surface, (60, 40, 30), (px, sho_y + int(10 * s), int(10 * s), int(10 * s))
        )
        pygame.draw.line(
            surface,
            (30, 20, 20),
            (px, sho_y + int(10 * s)),
            (px + int(10 * s), sho_y + int(10 * s)),
            1,
        )
    pygame.draw.rect(
        surface,
        C_DARK_BLUE,
        (cx + int(8 * s) * d, sho_y + int(14 * s), int(10 * s), int(10 * s)),
    )
    pygame.draw.rect(
        surface,
        (200, 200, 200),
        (cx + int(8 * s) * d, sho_y + int(14 * s), int(10 * s), int(10 * s)),
    )
```

```
[illegible]
```

[illegible]

[illegible]

[illegible]

```
                                        wake_alpha = int(16 + (0.8 - min(0.8, s / 3.4)) * 40)
    hull = [
        (hx - 18 * s, hy + 10 * s),
        (hx + 8 * s, hy + 8 * s),
        (hx + 24 * s, hy + 10 * s),
        (hx + 18 * s, hy + 18 * s),
        (hx - 18 * s, hy + 18 * s),
    ]
    pygame.draw.polygon(surface, hull_dark, hull)
    pygame.draw.polygon(surface, (10, 16, 18), hull, 1)
    deck = [
        (hx - 16 * s, hy + 8 * s),
        (hx + 14 * s, hy - 1 * s),
        (hx + 17 * s, hy + 7 * s),
        (hx - 8 * s, hy + 10 * s),
    ]
    pygame.draw.polygon(surface, hull_mid, deck)
    cabin = pygame.Rect(int(hx - 6 * s), int(hy - 10 * s), int(10 * s), int(10 * s))
    pygame.draw.rect(surface, hull_mid, cabin, border_radius=max(1, int(2 * s)))
    pygame.draw.rect(surface, hull_lo, cabin, 1, border_radius=max(1, int(2 * s)))
    win = pygame.Rect(int(hx + 1 * s), int(hy - 7 * s), int(10 * s), int(3 * s))
    pygame.draw.rect(surface, (40, 60, 58), win, border_radius=1)
    mast_x = int(hx + 8 * s)
    mast_y = int(hy - 10 * s)
    pygame.draw.line(surface, (20, 30, 40), (mast_x, mast_y), (mast_x + int(4 * s), mast_y - int(14 * s)), 1)
    if elapsed // 40 % 2 == 0:
        pygame.draw.circle(surface, (220, 60, 50), (mast_x + int(4 * s), mast_y - int(14 * s)), max(1, int(1 * s)))
    nose_flash = False
    for launch in (ATK_MISSILE_1_LAUNCH, ATK_MISSILE_2_LAUNCH, ATK_MISSILE_3_LAUNCH):
        if launch <= elapsed < launch + 10:
            nose_flash = True
            break
    if nose_flash:
        flash_pos = (int(hx + 14 * s), int(hy + 8 * s))
        for r in (int(10 * s), int(6 * s), int(3 * s)):
            pygame.draw.circle(surface, (255, 230, 160, 180), flash_pos, max(1, r))
    for i in range(3):
        ww = int((30 - i * 6) * s)
        wake = pygame.Surface((max(8, ww), max(2, int(3 * s))), pygame.SRCALPHA)
        pygame.draw.ellipse(wake, (220, 160, 190, max(10, wake_alpha - i * 8)), wake.get_rect())
        wx = int(hx - (18 + i * 14) * s)
        wy = int(hy + (16 + math.sin(elapsed * 0.05 + i * 0.7) * 2) * s)
        surface.blit(wake, (wx, wy))
def _draw_attack_helicopter(surface, elapsed):
    """A very distant helicopter silhouette crossing the horizon."""
    if elapsed < ATK_[illegible]:
        return
    [illegible]
```

[illegible]

```
pygame.draw.ellipse(ripple, (200, 170, 80, alpha), rect, 1)
    surface.blit(ripple, (int(cx - rw // 2), int(waterline - rh // 2 + 10 * s)))
duck_start_y = waterline - 300 * s
# -- Collapsing legs and duck ---------------------------------
if collapse > 0.005:
    if duck_mode == "topple":
        # Center rig: legs fold inward and duck drops almost vertically.
        duck_y = duck_start_y + (waterline - 56 * s - duck_start_y) * (collapse ** 0.78)
        duck_w = (310 - 70 * collapse) * s
        duck_h = (90 + 54 * collapse) * s
        twist = math.sin(t * 0.02) * 7 * s
        leg_roots = [cx - 107 * s, cx - 35 * s, cx + 35 * s, cx + 107 * s]
        for i, root_x in enumerate(leg_roots):
            side = -1 if root_x < cx else 1
            top_x = cx + (-84 + i * 57) * s * (1.0 - 0.44 * collapse)
            top_y = duck_y + (18 + 31 * 0.8) * s
            knee_x = cx + side * (58 + i % 2 * 20) * s * (1.0 - 0.35 * twist)
            knee_y = waterline - (58 - 15 * i) * s + collapse * 44 * s
            pygame.draw.line(surface, (35, 30, 28), (int(top_x), int(top_y)), (int(knee_x), int(knee_y)), max(3, int(9 * s)))
            pygame.draw.line(surface, (35, 30, 28), (int(knee_x), int(knee_y)), (int(root_x), int(waterline + 20 * s)), max(3, int(9 * s)))
        # Two duck halves collapsing toward the middle, with a visible crack.
        for half, side in enumerate((-1, 1)):
            half_w = duck_w * 0.54
            center_x = cx + side * duck_w * 0.23 * (1.0 - collapse)
            dip = (14 + 40 * collapse) * s
            pts = [
                (center_x - side * half_w * 0.78, duck_y - duck_h * 0.40),
                (cx + side * 4 * s + twist, duck_y - duck_h * 0.48 + dip),
                (cx + side * 14 * s + twist, duck_y + duck_h * 0.35 + dip),
                (center_x - side * half_w * 0.45, duck_y + duck_h * 0.38),
            ]
            pygame.draw.polygon(surface, (37, 30, 27), pts)
            pygame.draw.polygon(surface, (70, 60, 50), pts, 3)
        pygame.draw.line(surface, (255, 245, 40, int(200 * twist)), (int(cx), int(duck_y - 40 * s)), (int(cx), int(duck_y + 30 * s)), max(2, int(3 * s)))
        # Beak slumps downward into the hole.
        beak_base = (cx + twist, duck_y - 8 * s)
        beak_tip = (cx + math.sin(t * 0.08) * 10 * s, duck_y - (64 - 70 * collapse) * s)
        pygame.draw.line(surface, (30, 40, 50), (int(beak_base[0]), int(beak_base[1])), (int(beak_tip[0]), int(beak_tip[1])), max(2, int(4 * s)))
    else:
        # Left/right rigs: duck topples, but right halfs harder and tears apart.
        duck_y = duck_start_y + (waterline - 58 * s - duck_start_y) * (collapse ** (0.62 if style == 0 else 0.88))
        duck_w = (310 - (30 if style == 0 else 80) * collapse) * s
        duck_h = (90 - 8 * collapse) * s
```

```
[illegible]
```

```
[illegible]
```

```
[illegible]
```

```
# Splashes on top of rain.
    if _pixie.splashes:
        splash_surf = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
        for s in _pixie.splashes:
            t = s["age"] / s["max_age"]
            r = int(s["radius"] * (0.5 + t * 1))
            a = int(180 * (1 - t))
            if a <= 0 or r <= 0:
                continue
            pygame.draw.ellipse(
                splash_surf, (180, 200, 230, a),
                (int(s["x"] - r), int(s["y"] - r // 4), r * 2, r),
                1,
            )
        surface.blit(splash_surf, (0, 0))


def _draw_embers(surface):
    """Drifting fire embers from the right-side bg glow."""
    if not _pixie.initialized or not _pixie.embers:
        return
    em_surf = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    for e in _pixie.embers:
        t = e["age"] / e["max_age"]
        # Brightness fades in then out.
        fade = math.sin(t * math.pi) * e["brightness"]
        if fade <= 0:
            continue
        # Colour: bright yellow at birth → orange mid → red at death.
        if t < 0.4:
            col = (255, int(200 + 40 * (1 - t / 0.4)), 80, int(220 * fade))
        elif t < 0.75:
            col = (255, 150, 40, int(200 * fade))
        else:
            col = (200, 80, 30, int(180 * fade))
        r = e["size"]
        pygame.draw.circle(em_surf, col, (int(e["x"]), int(e["y"])), r)
        # Soft halo around bright embers.
        if fade > 0.6:
            halo_a = int(60 * fade)
            pygame.draw.circle(
                em_surf, (255, 140, 40, halo_a),
                (int(e["x"]), int(e["y"])), r * 3, 1
            )
    surface.blit(em_surf, (0, 0))


def _draw_vignette(surface):
    """Pre-rendered radial dark edge overlay."""
    if _pixie.vignette is not None:
        surface.blit(_pixie.vignette, (0, 0))


def _draw_title_glow(surface, title_x, title_y, title_w, title_h):
    """Slow orange glow pulse behind the title."""
    if not _pixie.initialized:
        return
    # Glow disc covers the title plus a 40px margin.
    glow_w = title_w + 80
    glow_h = title_h + 60
    glow = pygame.Surface((glow_w, glow_h), pygame.SRCALPHA)
    cx, cy = glow_w // 2, glow_h // 2
    max_r = math.hypot(cx, cy)
    pulse = _pixie.title_glow_pulse
    # Layered concentric ellipses, each softer than the last.
    for r in range(int(max_r), 0, -6):
        t = r / max_r
        eased = (1 - t) ** 2
        alpha = int(35 * eased * pulse)
        if alpha < 2:
            continue
        pygame.draw.ellipse(
            glow, (255, 140, 0, alpha),
            (cx - r, cy - int(r * 0.55), r * 2, int(r * 1.1))
        )
    surface.blit(
        glow,
        (title_x + title_w // 2 - glow_w // 2,
         title_y + title_h // 2 - glow_h // 2),
        special_flags=pygame.BLEND_RGBA_ADD,
    )


def _draw_scanline_drift(surface, x, y, w, h):
    """A bright horizontal scanline that slowly drifts across the title."""
    if not _pixie.initialized:
        return
    # Drift period: 5 seconds top-to-bottom.
    period = 5.0
    t = (_pixie.t % period) / period
    line_y = y + int(t * h)
    # Bright line + soft glow above/below.
    line_surf = pygame.Surface((w, 12), pygame.SRCALPHA)
    for i in range(12):
        a = int(90 * (1 - abs(i - 6) / 6))
        if a > 0:
            pygame.draw.line(line_surf, (255, 200, 140, a),
                             (0, i), (w, i), 1)
    surface.blit(line_surf, (x, line_y - 6),
                 special_flags=pygame.BLEND_RGBA_ADD)


def _draw_boot_banner(surface):
    """Typewriter COMPACT boot sequence shown on first menu show.

    Returns True while the boot is still playing (so draw_menu can suppress
    the normal classification banner), False when done.
    """
    if _pixie.boot_done:
        return False

    # Backdrop — same dimensions as the normal classification banner.
    banner = pygame.Surface((440, 80), pygame.SRCALPHA)
    banner.fill((0, 10, 20, 230))
    pygame.draw.line(banner, C_ORANGE, (0, 0), (440, 0), 1)
    pygame.draw.line(banner, C_ORANGE, (0, 79), (440, 79), 1)
    surface.blit(banner, (20, 20))

    # Banner lines with a typewriter effect, staggered by _BOOT_LINE_DELAY.
    chars_remaining = _pixie.boot_chars_typed
    y_offset = 28
    for line_idx, line in enumerate(_BOOT_LINES):
        # Each line starts typing at line_idx * _BOOT_LINE_DELAY worth of
        # chars consumed.
        start_chars = int(line_idx * _BOOT_LINE_DELAY * _BOOT_CHAR_RATE)
        if chars_remaining <= start_chars:
            break
        chars_on_this_line = min(
            len(line), chars_remaining - start_chars
        )
        visible = line[:chars_on_this_line]
        # Blinking cursor on the line currently being typed.
        if chars_on_this_line < len(line) and (
           line_idx == len(_BOOT_LINES) - 1 or
           chars_remaining < start_chars + len(line)):
            if int(_pixie.t * 4) % 2 == 0:
                visible += "_"
        surf = _classification_font.render(visible, True, C_ORANGE)
        surface.blit(surf, (30, y_offset))
        y_offset += 14
        if y_offset > 90:  # clamp to banner area
            break

    return True


def _draw_news_ticker(surface):
    """Scrolling news ticker above the footer."""
    # Compose the full ticker string with separators.
    separator = "  //  "
    full = separator.join(_TICKER_ITEMS) + separator
    full_surf = _footer_font.render(full, True, (180, 160, 140))
    sep_surf = _footer_font.render(separator, True, C_ORANGE)
    sep_w = sep_surf.get_width()
    full_w = full_surf.get_width()

    # Ticker sits above the footer divider line, full width of the dark panel.
    ticker_y = HEIGHT - 56
    ticker_h = 18
    # Backdrop strip.
    strip = pygame.Surface((WIDTH, ticker_h), pygame.SRCALPHA)
    strip.fill((0, 0, 0, 140))
    surface.blit(strip, (0, ticker_y))

    # COMPACT WIRE label on the left — static.
    label = _classification_font.render("COMPACT WIRE", True, C_ORANGE)
    label_w = label.get_width() + 16
    surface.blit(label, (8, ticker_y + 3))
    # Vertical separator after the label.
    pygame.draw.line(surface, (90, 60, 30),
                     (label_w, ticker_y),
                     (label_w, ticker_y + ticker_h), 1)

    # Scrolling text window from label_w to WIDTH.
    window_w = WIDTH - label_w - 8
    if window_w <= 0 or full_w <= 0:
        return

    # Offset wraps modulo (full_w + sep_w) so the loop is seamless.
    period = full_w + sep_w
    offset = int(_pixie.ticker_offset) % period

    # Draw the text twice — once at -offset, once at (period - offset) — so
    # the loop has no visible gap when it wraps.
    clip_rect = pygame.Rect(label_w + 4, ticker_y, window_w, ticker_h)
    prev_clip = surface.get_clip()
    surface.set_clip(clip_rect)
    surface.blit(full_surf, (label_w + 8 - offset, ticker_y + 3))
    surface.blit(full_surf, (label_w + 8 - offset + period, ticker_y + 3))
    surface.set_clip(prev_clip)

    # Top border line on the ticker strip.
    pygame.draw.line(surface, (90, 60, 30),
                     (0, ticker_y), (WIDTH, ticker_y), 1)

# ── Keyboard navigation API ────────────────────────────────
# Public functions called by scratchin.py. Designed so the existing mouse
# click path keeps working unchanged — keyboard input is layered on top.

def _visible_button_indices():
    """Return list of indices into `buttons[]` that are currently on-screen.

    draw_menu() sets each button's `y` to -999 when it should be hidden
    (e.g. CONTINUE when no save exists). We respect that convention here.
    """
    return [i for i, b in enumerate(buttons) if b.y > 0]

def handle_menu_keydown(event):
    """Process a KEYDOWN event in the MENU state.

    Call from scratchin.py's event loop when game_state == "MENU".
    Returns None — side effects are stored in _pixie.pending_trigger, which
    scratchin.py reads via consume_keyboard_trigger().
    """
    if not _pixie.initialized:
        _init_menu_anim()

    if event.key in (pygame.K_UP, pygame.K_w):
        visible = _visible_button_indices()
        if not visible:
            return
        if _pixie.keyboard_idx == -1:
            # Transitioning from mouse control — pick the bottom visible.
            _pixie.keyboard_idx = visible[-1]
        elif _pixie.keyboard_idx in visible:
            i = visible.index(_pixie.keyboard_idx)
            _pixie.keyboard_idx = visible[(i - 1) % len(visible)]
        else:
            _pixie.keyboard_idx = visible[-1]
        audio.play("menu_hover", volume=0.3)
    elif event.key in (pygame.K_DOWN, pygame.K_s):
        visible = _visible_button_indices()
        if not visible:
            return
        if _pixie.keyboard_idx == -1:
            _pixie.keyboard_idx = visible[0]
        elif _pixie.keyboard_idx in visible:
            i = visible.index(_pixie.keyboard_idx)
            _pixie.keyboard_idx = visible[(i + 1) % len(visible)]
        else:
            _pixie.keyboard_idx = visible[0]
        audio.play("menu_hover", volume=0.3)
    elif event.key in (pygame.K_RETURN, pygame.K_SPACE):
        if _pixie.keyboard_idx != -1:
            _pixie.pending_trigger = _pixie.keyboard_idx
            buttons[_pixie.keyboard_idx].press()
            audio.play("menu_click", volume=0.5)

def consume_keyboard_trigger():
    """Return the button index that should fire this frame, or -1.

    scratchin.py should call this after draw_menu() and route the trigger
    through the same action handlers as a mouse click.
    """
    t = _pixie.pending_trigger
    _pixie.pending_trigger = -1
    return t

def reset_keyboard_selection():
    """Clear keyboard focus — called when the mouse moves."""
    _pixie.keyboard_idx = -1

# ──────────────────────────────────

# SERVER BROWSER

# ──────────────────────────────────

server_list = [

    ("Pixie Dough's SCRATCHIN 01 Launch Party!!", 3, 4, 45),

    ("Damin Palace [RUSSIA] [NO BOTS]", 6, 16, 140),

    ("Sebi's Parlament [EF] [HARDCORE ONLY]", 14, 16, 30),

    ("GDM Archives // LEAKED", 0, 16, 85),

    ("Putin's Summer House [VIP ONLY]", 2, 8, 400),

    ("[redacted] [REDACTED]", 7, 7, 150),

    ("Vasiliu's Cigarette Break", 3, 4, 60),

    ("Mitchel's Safe House [POLS: TSO]", 0, 4, 20),

    ("FBA Recruitment Center", 10, 24, 80),

    ("Graystone OS Official", 0, 64, 250),

    ("noobs only pls", 3, 99, 120),

    ("[24/7] OIL RIG EXPLOSION DM", 12, 16, 75),

    ("RU-BUDET KOSHKA WANT CONNECTED", 29, 100, 35),

    ("yo mama's basement", 1, 4, 8),

    ("FREE V-BUCKS REAL NOT FAKE", 4, 4, 180),

]

# Server rules — shown on hover

SERVER_RULES = {

    "Pixie Dough's SCRATCHIN 01 Launch Party!!": [

        "GAMEMODE:  Launch Party Deathmatch",

        "MAP:       Dumpster HQ Rooftop",

        "RULES:     1. Bring your own fireworks.",

        "           2. No actual Dough allowed.",

        "           3. Pixie Dough always wins.",

    ],

    "Damin Palace [RUSSIA] [NO BOTS]": [

        "GAMEMODE:  Siege for Gold",

        "MAP:       The Palace",

        "RULES:     Literally none. We tried.",

        "           Last admin resigned in tears.",

    ],

    "Sebi's Parlament [EF] [HARDCORE ONLY]": [

        "GAMEMODE:  Parliamentary Procedure DM",

        "MAP:       Parliament Chamber",

        "RULES:     1. Address the chair properly.",

        "           2. No filibustering past 2am.",

        "           3. Bring snacks for the recess.",

    ],

    "GDM Archives // LEAKED": [

        "GAMEMODE:  ???",

        "MAP:       ???",

        "RULES:     Do not discuss what you see here.",

        "           Its not, its old system tests.",

    ],

    "Putin's Summer House [VIP ONLY]": [

        "GAMEMODE:  Bear wrestling",

        "MAP:       Dacha + Sauna",

        "RULES:     1. No shirts allowed.",

        "           2. Stay away from the windows.",

        "           3. Compliment the host's judo.",

    ],

    "[redacted] [REDACTED]": [

        "GAMEMODE:  [redacted]",

        "MAP:       [redacted]",

        "RULES:     [redacted]",

        "           [redacted]",

    ],

    "Vasiliu's Cigarette Break": [

        "GAMEMODE:  Standing Around",

        "MAP:       Behind the Warehouse",

        "RULES:     1. No talking shop.",

        "           2. Share the lighter.",

        "           3. Back to work in 5 minutes.",

    ],

    "Mitchel's Safe House [POLS: TSO]": [

        "GAMEMODE:  Cooperative Stealth",

        "MAP:       Safe House Interior",

        "RULES:     1. Keep your voice down.",

        "           2. Do not turn on the lights.",

        "           3. The code is on the desk.",

    ],

    "FBA Recruitment Center": [

        "GAMEMODE:  Capture the Flag",

        "MAP:       Glow District",

        "RULES:     1. Free graffiti supplies provided.",

        "           2. No GBAF affiliation.",

        "           3. Make Sabi YOU.",

    ],

    "Graystone OS Official": [

        "GAMEMODE:  Peacekeeping Simulator",

        "MAP:       Districts-wide",

        "RULES:     1. Follow protocol at all times.",

        "           2. Paperwork must be in triplicate.",

        "           3. Do not ask about the ads.",

    ],

    "noobs only pls": [

        "GAMEMODE:  Training Grounds",

        "MAP:       Tutorial Map",

        "RULES:     1. No tryhards.",

        "           2. Be nice to new players.",

        "           3. We were all noobs once.",

    ],

    "[24/7] OIL RIG EXPLOSION DM": [

        "GAMEMODE:  Demolition Derby",

        "MAP:       Offshore Drone Platform",

        "RULES:     1. Everything must explode.",

        "           2. Extinguish 5 minutes for CEO.",

        "           3. No fishing during gameplay.",

    ],

    "RU-BUDET KOSHKA WANT CONNECTED": [

        "GAMEMODE:  Subscribe Battle Royale",

        "MAP:       Mystery Warehouse",

        "RULES:     1. Must subscribe to enter.",

        "           2. Look out with cigarettes.",

        "           3. Not actually 99bouse.",

    ],

    "yo mama's basement": [

        "GAMEMODE:  LAN Party",

        "MAP:       The Basement",

        "RULES:     1. Bring your own Mountain Dew.",

        "           2. Pew paid pizza at 8.",

        "           3. No yelling, she's sleeping.",

    ],

    "FREE V-BUCKS REAL NOT FAKE": [

        "GAMEMODE:  TOTALLY LEGIT GIVEAWAY",

        "MAP:       N/A",

        "RULES:     Just enter your card details.",

        "           100% legit. We promise.",

    ],

}

# Region rotation for flavor

SERVER_REGIONS = [

    "NORTH-ZONE-EAST (Lobby)",

    "SOUTHERN-NORTH (Lobby)",

    "NORTH-ZONE-CENTRAL (Lobby)",

    "GLOW-DISTRICT main",

    "[UNKNOWN REGION]",

]

# Server-specific kick messages — falls back to global pool

KICK_MESSAGES_BY_SERVER = {

    "Sebi's Parlament [EF] [HARDCORE ONLY]": [

        "Kicked: lacked sufficient bureaucratic forms",

        "Disconnected: your parliamentary immunity has expired",

        "Kicked: failed to address the chair properly",

    ],

    "GDM Archives // LEAKED": [

        "Connection severed by external party",

        "Banned: We supposedly Remove endpoints.",

        "You were never here.",

    ],

    "Putin's Summer House [VIP ONLY]": [

        "Kicked: insufficient loyalty score",

        "Disconnected: a man fell out of a window",

        "Kicked: refused to take shirt off",

    ],

    "[redacted] [REDACTED]": [

        "[redacted]",

        "[redacted]",

    ],

    "RU-BUDET KOSHKA WANT CONNECTED": [
```

[illegible]